

碳化硅外延层厚度的确定

摘要

本题针对红外干涉法确定晶圆外延层厚度的问题，建立了从两光束干涉到多光束干涉递进的模型，并给出相应的厚度求解算法与多光束干涉判别方法，依次应用于碳化硅与硅两种晶圆片的实测光谱。

针对问题一，在界面只有一次反射、透射的情形下，把两束反射光的复振幅叠加，得到随相位余弦变化的反射率。条纹幅度与条纹位置分别携带界面反射信息与相位信息：由峰谷反射率提取包络线，按菲涅耳关系反推外延层折射率，即**包络线法**；由峰谷位置反推相位，结合折射率解出外延层厚度。求解给出两个方案：方案一对两种偏振取平均近似并作**非线性最小二乘全谱拟合**，方案二取**垂直入射近似**得到解析公式。

针对问题二，把问题一的模型在碳化硅晶圆片的实测光谱上实现，并分析结果的可靠性。先对实测反射谱做预处理，剔除反射率非正的异常点；再识别干涉峰谷，由峰谷包络逐对反推折射率，用文献折射率范围逐段筛选谱段；随后经**线性插值**得到随波数变化的折射率，用相邻峰谷法按相位条件逐对解出一组厚度估计值，取中位数作为厚度结果；最后用**非线性最小二乘的全谱拟合**与两个入射角的结果交叉验证可靠性。三类结果彼此相符，最终得到**碳化硅外延层厚度约 $7.48\ \mu\text{m}$** 。

针对问题三，先把光在膜内多次往返产生的各束反射光按等比数列求和，推得多光束干涉的**反射率表达式**，由此给出产生的两个必要条件：相邻出射光束的振幅比不可过小，相邻光束的光程差不超过光源的相干长度；再与两光束表达式对比，分析对厚度计算精度的影响：多光束干涉不改变条纹周期，却使谷变尖变深、峰变宽变平缓，包络线法因此失效。在此基础上设计判别，以振幅比估计、中线占比与**傅里叶功率谱谐波**三种证据相互印证，对四份附件逐一判别：**碳化硅判定不存在多光束干涉**，问题二结果不受影响；**硅晶圆片判定存在多光束干涉**，厚度计算中折射率改取文献名义值，用以**Drude介电函数**描述衬底色散的非线性最小二乘全谱拟合与计入相移修正的相邻峰谷法两种算法求解，再以傅里叶功率谱独立检验，得**硅外延层厚度约 $3.39\ \mu\text{m}$** 。

综上，本文以条纹幅度与位置信息的分离为主线，把模型由两光束递进至多光束：问题一建立基本模型，问题二在实测光谱上实现并验证其可靠性，问题三给出模型的适用边界与多光束影响下的修正方法，可为同类半导体外延层厚度的无损测量提供参考。

关键字： 红外干涉法 包络线法 相邻峰谷法 多光束干涉判别 非线性最小二乘拟合 傅里叶检验

一、问题重述

1.1 问题背景

碳化硅是一种综合性能优越的第三代半导体材料，其外延层的厚度是外延材料的关键参数之一，对器件性能有重要影响。制定一套科学、准确、可靠的外延层厚度测试方法，具有重要的工程意义。

红外干涉法是外延层厚度测量的无损测量方法。外延层与衬底因掺杂载流子浓度的不同而有不同的折射率，红外光入射后，一部分从外延层表面反射出来，另一部分在外延层与衬底的界面反射回来，两束光在一定条件下产生干涉条纹。根据红外光谱的波长、外延层的折射率和红外光的入射角等参数，即可确定外延层的厚度。

通常外延层的折射率不是常数，它与掺杂载流子的浓度、红外光谱的波长等参数有关。光波在两界面间还可能发生多次反射与透射，产生多光束干涉，进而影响厚度的测定。本文围绕上述三个层次，依次建立两光束干涉模型、设计厚度求解算法，并研究多光束干涉的判别与消除。

1.2 需要解决的问题

本文需要解决以下三个问题。

问题一：考虑外延层和衬底界面只有一次反射、透射所产生干涉条纹的情形，建立确定外延层厚度的数学模型。

问题二：根据问题一的数学模型，设计确定外延层厚度的算法。对附件 1 和附件 2 提供的碳化硅晶圆片实测光谱（入射角分别为 10° 和 15° ，第 1 列为波数，第 2 列为反射率）给出计算结果，并分析结果的可靠性。

问题三：光波在外延层界面和衬底界面可以产生多次反射和透射，从而产生多光束干涉。推导产生多光束干涉的必要条件，分析多光束干涉对外延层厚度计算精度可能产生的影响。根据必要条件，分析附件 3 和附件 4 提供的硅晶圆片测试结果（入射角分别为 10° 和 15° ）是否出现多光束干涉，给出确定硅外延层厚度的数学模型、算法及相应的计算结果；若认为多光束干涉也会出现在碳化硅晶圆片的测试结果中并影响厚度计算的精度，则设法消除其影响，并给出消除影响后的计算结果。

二、问题分析

2.1 问题一的分析

问题一要求在界面只有一次反射、透射的情形下建立确定外延层厚度的模型。入射光在外延层表面与衬底界面先后反射，两束光叠加形成干涉，反射率取决于两束光的相位差。难点在于相位差把折射率与厚度耦合在一起，两个量都未知。但条纹的幅度只由界面的反射与透射决定，条纹的位置只由光在外延层内的光程决定，因此可以先由峰谷反射率提取包络，按菲涅耳关系反推出折射率，再由相位确定厚度。折射率随波数变化，包络提取与相位反推都需逐峰谷对局部进行，两条路径分离又可相互检验。

2.2 问题二的分析

问题二要求把问题一的模型在实测光谱上实现，并分析结果的可靠性。数据的难点有三处：实测反射率含异常读数，峰谷识别受噪声干扰，碳化硅的折射率又随波数变化。先剔除异常点，再识别峰谷；每一对相邻峰谷按相位条件给出一个相互独立的厚度估计，多个估计取中位数作为结果，以离散程度衡量可靠性。折射率的色散使常数折射率近似只在部分谱段成立，需用文献名义值逐段核查，只让近似成立的谱段参与统计。可靠性还需要独立验证：同一晶圆片不同入射角的测量结果应当接近；再用全谱拟合从整条谱线的形状求厚度，与逐对峰谷的统计对照。几种结果彼此一致，厚度才可信。

2.3 问题三的分析

问题三的任务有两层：先推导多光束干涉产生的必要条件，再用它判别实测数据并计算厚度。光在膜内多次往返时，每一束后续光比前一束多经历一次界面反射与膜内衰减，衰减的快慢决定后续光束能否参与叠加；各束光要稳定干涉，相邻光束的光程差还须不超过光源的相干长度。判别的难点在于界面参数无法直接测量，且多光束干涉不改变条纹的周期、只改变条纹的形状，从周期上看不出来，只能从形状与幅度入手：振幅比可由峰谷反射率反推，条纹形状可由数据点相对中线的分布统计刻画，频域结构则反映谐波成分。一旦确认存在多光束干涉，包络线法失效，折射率需改取文献名义值，厚度可改由多光束模型全谱拟合、计入相移修正的峰谷法与傅里叶检验相互印证地求解；若判定不存在多光束干涉，问题二的算法仍然适用。

三、模型假设

1. 外延层为均匀介质，晶圆表面平整，两个界面光滑且相互平行。
2. 不同波数的光谱成分互不影响。
3. 测量过程中晶圆的光学性质稳定。

4. 环境温度与空气对光传播的影响可以忽略。

四、符号说明

表 1 符号说明

符号	说明
σ	波数
θ	入射角
$n(\sigma)$	外延层折射率
n_0	空气折射率
d	外延层厚度
$\dot{E}$	复电场振幅（相量）
ϕ	两束反射光的相位差
R	反射率
$R_{\max}$ 、 $R_{\min}$	干涉峰、谷的反射率
r_{01} 、 r_{12}	空气—外延层、外延层—衬底界面的反射系数
A	光在外延层内往返一次的衰减因子
B	第二束反射光的振幅系数
ρ	相邻出射光束的振幅比
L_c	光源的相干长度

五、模型的建立与求解

5.1 问题一的模型建立

5.1.1 问题分析

题目要求，在只考虑外延层和衬底界面的一次反射、透射所产生的干涉条纹的情形下，建立确定外延层厚度的数学模型。

光源以一定入射角入射外延层表面时，一部分光在表面直接反射，记为**反射光 1**；另一部分光折射进入外延层，在衬底界面反射一次后，再从外延层表面折射而出，记为**反射光 2**。

实际光源具有一定的宽度，并非理想的一束光。如下图所示，反射光 1 与反射光 2 可能相互叠加，产生干涉。

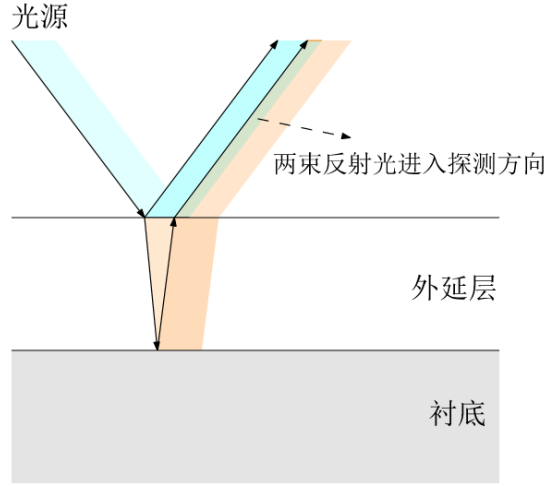


图 1 光源叠加示意图

两束光叠加后的反射光强随波数呈明暗交替的振荡，即干涉条纹。条纹由两个因素共同决定：一是光在传播、折射与反射过程中的强度损耗，它决定两束光各自的振幅，从而决定条纹的幅度；二是两束光的相位差，它决定叠加是加强还是减弱，从而决定条纹的位置。

这两个因素分别携带不同的信息，求解可以分为两步：先由条纹的幅度反推两束光的振幅，进而得到外延层折射率；再由条纹的位置计算外延层厚度。下面依次给出这两步的定量关系。

5.1.2 求解折射率

为便于叙述，以下公式的下标中，0 表示空气，1 表示外延层，2 表示衬底。

光具有波的性质。为同时刻画光波的振幅与相位，引入复电场振幅（相量） $\dot{E}$ 。下面分别分析反射光 1 与反射光 2 的光强损耗。设光源的初始复电场振幅为 $\dot{E}_0$ 。

对于反射光 1，它在空气—外延层上表面直接反射，强度有所损耗。设该界面的反射系数为 r_{01} ，则：

$$\dot{E}_1 = r_{01}\dot{E}_0 \quad (1)$$

对于反射光 2，它依次经历以下过程：（一）从空气透射进入外延层；（二）在外延层中传播到衬底界面；（三）在外延层-衬底界面反射；（四）再次穿过外延层；（五）从外延层透射回空气。设第一次透射系数为 t_{01} ，外延层—衬底界面的反射系数为 r_{12} ，第二次透射系数为 t_{10} ，光在外延层中往返一次的衰减为 A ，光 1 与光 2 的相位差为 ϕ 。工作波段内外延层与衬底的吸收较弱，各菲涅耳振幅系数均取实数，材料吸收与传播损耗统一由衰减因子 A 描述，则：

$$\dot{E}_2 = t_{01}r_{12}t_{10}A\dot{E}_0 \cdot e^{i\phi} \quad (2)$$

两束光叠加后的复电场振幅为：

$$\dot{E}_3 = \dot{E}_1 + \dot{E}_2 \quad (3)$$

叠加后的光强比值（反射率）为：

$$R = \left| \frac{\dot{E}_3}{\dot{E}_0} \right|^2 \quad (4)$$

把叠加后的反射率展开。记光 2 的振幅系数 $B = t_{01}r_{12}t_{10}A$ ，则：

$$R = |r_{01} + B e^{i\phi}|^2 = r_{01}^2 + B^2 + 2r_{01}B \cos \phi \quad (5)$$

式 (5) 中只有 $\cos \phi$ 随波数快速振荡， r_{01} 与 B 变化缓慢，在一个干涉条纹周期内可视为常数。因此，反射率的极值只由 $\cos \phi$ 决定： $\cos \phi$ 取 +1 时两束光同相叠加，反射率达到极大；取 -1 时反相叠加，反射率达到极小。由此得到反射率曲线的上、下包络线：

$$R_{\max} = (|r_{01}| + |B|)^2, \quad R_{\min} = (|r_{01}| - |B|)^2 \quad (6)$$

由上、下包络线反演薄膜光学常数与厚度的方法称为**包络线法**，本文在包络线的构造上参考了文献^[1]。

式 (6) 两端均为完全平方：反射率正比于合振幅的平方，而在极值处两束光恰好同相或反相，合振幅等于两束光振幅的和或差。对上、下包络开方，便得到两个可以直接相加减的振幅：

$$\sqrt{R_{\max}} = |r_{01}| + |B|, \quad \sqrt{R_{\min}} = ||r_{01}| - |B|| \quad (7)$$

这一步是包络线法的关键。若直接对 $R_{\max}$ 、 $R_{\min}$ 作和差，所得只是 $2(r_{01}^2 + B^2)$ 与 $4|r_{01}||B|$ ， $|r_{01}|$ 与 $|B|$ 仍以平方和乘积的形式耦合在一起，无法分离；开方之后，二者均以一次形式出现，可以分别解出：

$$|r_{01}| = \frac{\sqrt{R_{\max}} + \sqrt{R_{\min}}}{2}, \quad |B| = \frac{\sqrt{R_{\max}} - \sqrt{R_{\min}}}{2} \quad (8)$$

两个振幅的物理含义不同： r_{01} 是空气—外延层界面的反射系数，只由 n_0 、 n 、入射角和偏振状态决定，**与厚度 d 无关**，携带的是折射率信息； B 则集中了透射、界面反射与往返衰减的全部损耗。因此只需保留前者。记空气—外延层界面的反射系数大小为：

$$u(\sigma) = \frac{\sqrt{R_{\max}(\sigma)} + \sqrt{R_{\min}(\sigma)}}{2} \quad (9)$$

上述过程如图 2 所示。图中是一段由两光束模型生成的示意图：上图给出反射率 $R(\sigma)$ 的振荡曲线及其上、下包络线，并在某一波数处标出这一对极值；下图把两条包络开方。可见 $\sqrt{R_{\max}}$ 与 $\sqrt{R_{\min}}$ 关于 $u(\sigma)$ 对称分布，二者之和的一半即界面反射振幅 $|r_{01}|$ ，之差的一半即光 2 的振幅 $|B|$ 。这样，条纹的幅度信息经过以上三步，就被分解为两束光各自的振幅。

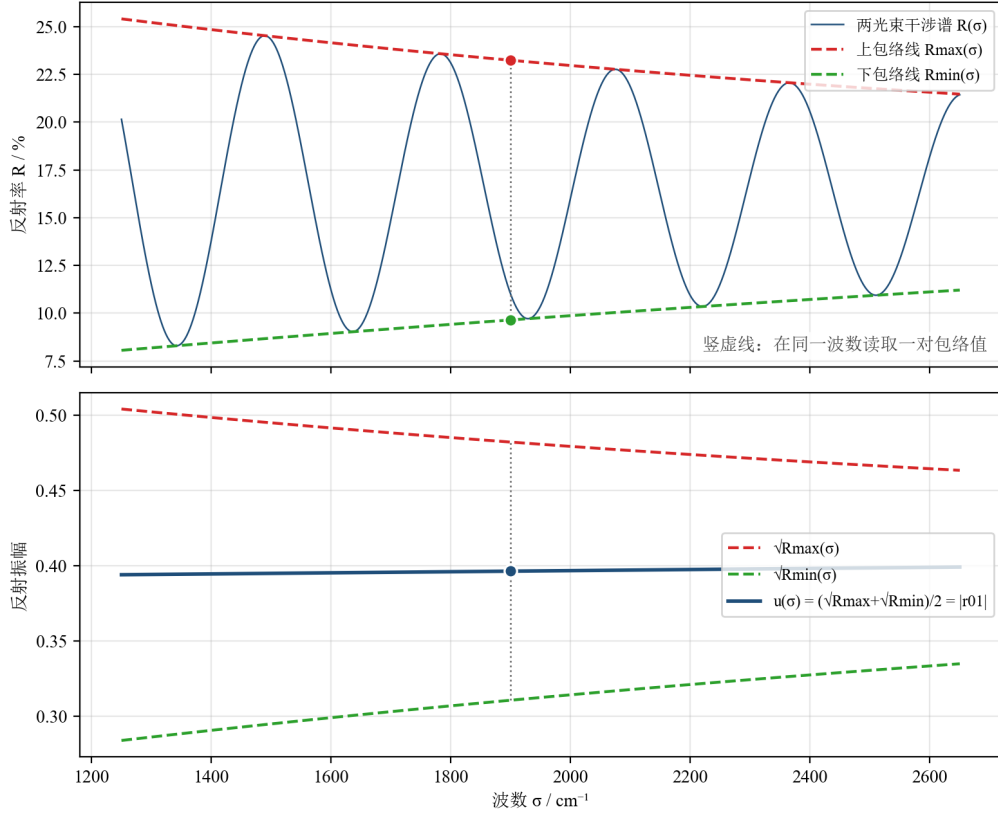


图 2 包络线法（由干涉条纹的幅度提取界面反射信息）

得到界面反射系数 $u(\sigma)$ 后，利用斜入射的界面反射公式反演折射率。外延层中的折射角由折射定律确定， $\sin \theta_1(\sigma) = \frac{n_0 \sin \theta}{n(\sigma)}$ ，即：

$$n(\sigma) \cos \theta_1(\sigma) = \sqrt{n^2(\sigma) - n_0^2 \sin^2 \theta} \quad (10)$$

若入射光为 s 偏振，斜入射时的振幅反射系数为：

$$r_{01}^{(s)}(\sigma) = \frac{n_0 \cos \theta - n(\sigma) \cos \theta_1(\sigma)}{n_0 \cos \theta + n(\sigma) \cos \theta_1(\sigma)} \quad (11)$$

若入射光为 p 偏振，振幅反射系数为：

$$r_{01}^{(p)}(\sigma) = \frac{n(\sigma) \cos \theta - n_0 \cos \theta_1(\sigma)}{n(\sigma) \cos \theta + n_0 \cos \theta_1(\sigma)} \quad (12)$$

接下来处理偏振。由式 (11)(12) 可见，只要入射角不为零， s 、 p 两种偏振的反射系数就不相等；而题目中的光源不区分偏振，探测器接收到的是两种偏振光强之和。若严格处理，需对两种偏振分别建立式 (5) 那样的双光束模型，分别计算反射率后相加。这样，实测峰谷包络中混入了两种偏振的贡献，无法由 $u(\sigma)$ 直接解出单一的 $|r_{01}|$ ，折射率也就没有显式解。对此给出两种处理方案。

方案一：平均近似。把式 (11)(12) 代入式 (5)，记两种偏振各自的反射率为 $R_s = |r_s|^2 + B^2 - 2|r_s|B \cos \phi$ 与 $R_p = |r_p|^2 + B^2 - 2|r_p|B \cos \phi$ （干涉项取负，是因为第二束光在衬底界面反射时存在与界面反射系数符号有关的额外相位，使反射率极大出现在 $\cos \phi = -1$ 处；该符号约定与 5.2.4 的实测全谱拟合一致）。对任意入射角 θ ，非偏振光的反射率取二者的算术平均：

$$R(\sigma, \theta) = \frac{R_s + R_p}{2} \quad (13)$$

方案二：垂直入射近似。当入射角较小时， $\cos \theta \approx \cos \theta_1 \approx 1$ ，代入式 (11)(12) 可知，两种偏振的反射系数退化为同一形式，即垂直入射的菲涅耳公式：

$$|r_{01}| = \frac{n(\sigma) - n_0}{n(\sigma) + n_0} \quad (14)$$

由此可以直接解出折射率的显式表达式：

$$n(\sigma) = n_0 \frac{1 + u(\sigma)}{1 - u(\sigma)} \quad (15)$$

两种方案的比较与选取在 5.2 节进行。

5.1.3 确定外延层厚度

上面已由峰谷包络线得到外延层的折射率 $n(\sigma)$ 。下面利用干涉条纹的位置信息，即相邻波峰、波谷的波数，确定厚度 d 。

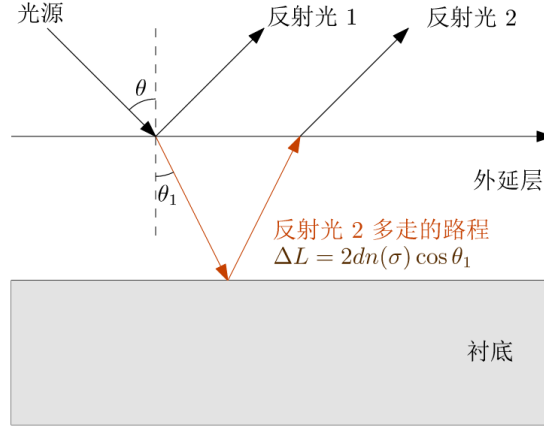


图 3 光路示意图

如上图所示，反射光 2 比反射光 1 多在外延层中往返一次，光程差为 $2dn(\sigma)\cos\theta_1$ ，对应的相位差为：

$$\phi = 4\pi\sigma dn(\sigma)\cos\theta_1 \quad (16)$$

由式 (5) 可知，波峰处 $\cos\phi = +1$ 、波谷处 $\cos\phi = -1$ ，峰、谷对应的相位差分别为 $2m\pi$ 与 $(2m+1)\pi$ 。因此，从一个波峰到相邻波谷，两束光的相位差改变半个周期：

$$4\pi d [\sigma_v n(\sigma_v) \cos\theta_{1,v} - \sigma_p n(\sigma_p) \cos\theta_{1,p}] = \pi \quad (17)$$

其中， p 、 v 分别代表波峰、波谷对应的参数。整理可得：

$$d = \frac{1}{4 |\sigma_v n(\sigma_v) \cos\theta_{1,v} - \sigma_p n(\sigma_p) \cos\theta_{1,p}|} \quad (18)$$

再代入式 (10) 消去折射角，得到**厚度与折射率的函数关系**：

$$d = \frac{1}{4 \left| \sigma_v \sqrt{n^2(\sigma_v) - n_0^2 \sin^2 \theta} - \sigma_p \sqrt{n^2(\sigma_p) - n_0^2 \sin^2 \theta} \right|} \quad (19)$$

式 (19) 中， σ_p 、 σ_v 由实测反射谱的峰谷位置读出， θ 由题目给出， $n(\sigma)$ 已由 5.1.2 求得，外延层厚度 d 即可算出。

5.1.4 总结

至此，确定外延层厚度所需的公式已全部给出。求解分为两步。

第一步，求折射率。方案 1，折射率满足以下方程：

$$R(\sigma, \theta) = \frac{R_s + R_p}{2} \quad (20)$$

其中 R_s 、 R_p 为两种偏振各自的反射率（见 5.1.2）。方案 2，折射率可以显化表示：

$$n(\sigma) = n_0 \frac{1 + u(\sigma)}{1 - u(\sigma)} \quad (21)$$

其中， $u(\sigma) = \frac{\sqrt{R_{\max}(\sigma)} + \sqrt{R_{\min}(\sigma)}}{2}$ 。

第二步，求厚度。把 $n(\sigma)$ 代入厚度公式：

$$d = \frac{1}{4 \left| \sigma_v \sqrt{n^2(\sigma_v) - n_0^2 \sin^2 \theta} - \sigma_p \sqrt{n^2(\sigma_p) - n_0^2 \sin^2 \theta} \right|} \quad (22)$$

其中 σ_p 、 σ_v 为相邻波峰、波谷的波数， θ 为入射角， n_0 为空气折射率。

5.2 问题二的模型建立与求解

5.2.1 问题分析

题目要求，在问题一建立的两光束干涉模型的基础上，设计确定碳化硅外延层厚度的算法，对附件 1（入射角 10° ）、附件 2（入射角 15° ）两块晶圆片的实测反射光谱给出厚度计算结果，并分析结果的可靠性。

附件 1、附件 2 是同一晶圆片在两种入射角下的独立实测光谱，数据第 1 列为波数，第 2 列为反射率。由问题一的分析，干涉条纹同时携带两层信息：**条纹位置**反映两束光的相位差，**条纹幅度**反映界面反射强度、只包含折射率 n 。因此，先从条纹幅度求出折射率 $n(\sigma)$ ，再把折射率代入条纹位置公式，即可解出厚度 d 。

算法流程如下图所示。

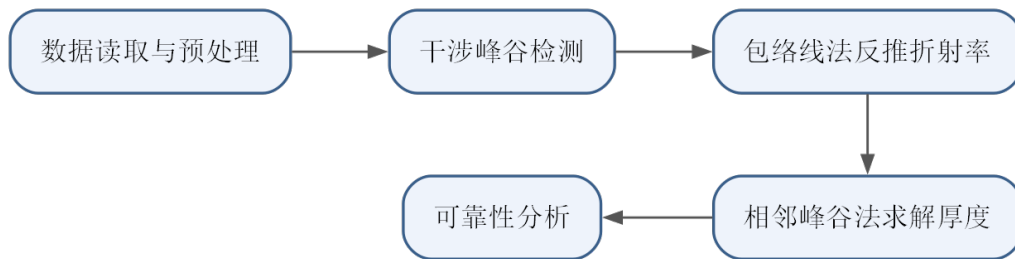


图 4 问题二算法流程图

其中，第 2、3 两步是核心：第 2 步由条纹幅度反推折射率，第 3 步由条纹位置求出厚度。

5.2.2 数据预处理

两份附件的原始反射光谱如下图所示。由图可见，反射率曲线在若干波段出现与干涉条纹正常形态不符的异常特征：部分波数处反射率跳变，谱线起始处还存在一个突跳的零值点。这些异常读数不反映真实的干涉信息，若直接参与后续计算，会污染折射率与厚度的估计。因此，在定量计算之前先对原始数据进行预处理。

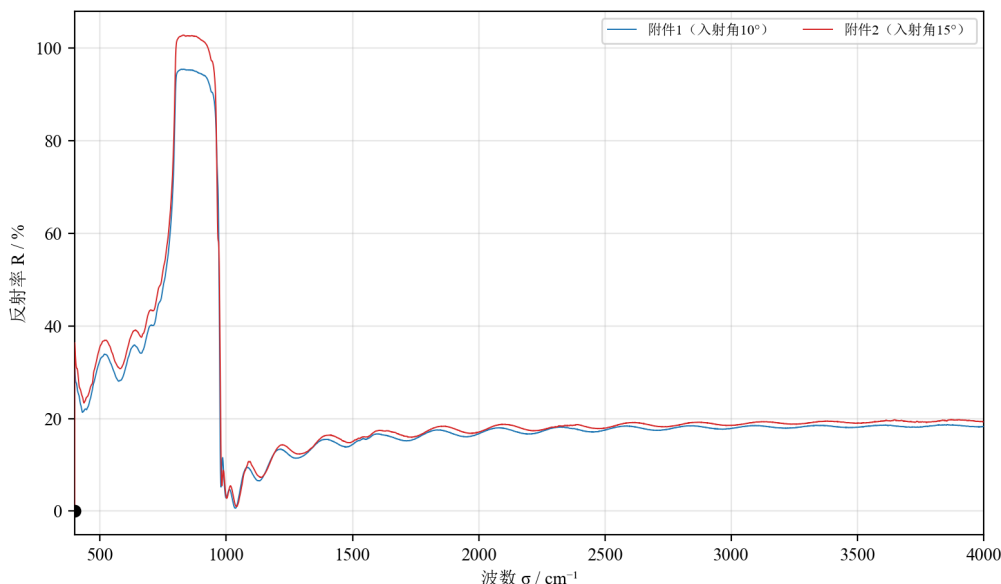


图 5 原始反射光谱

数据预处理主要包括以下两个方面：

1. **删除反射率为零或负的异常数据点。**反射率在物理上不可能取零值或负值， $R \leq 0$ 的点不可能是任何波段的真实读数。
2. **删除估算折射率超出理论值范围的数据。**查阅文献得，碳化硅在红外透明区的折射率范围为 $[2.20, 2.60]$ ^[2, 3]。由相邻极大值与极小值对应的波数反推折射率，去除超出理论范围的峰谷对。

删除异常点后的反射光谱如下图所示，红色三角与绿色倒三角分别标出保留峰谷对的峰、谷位置。

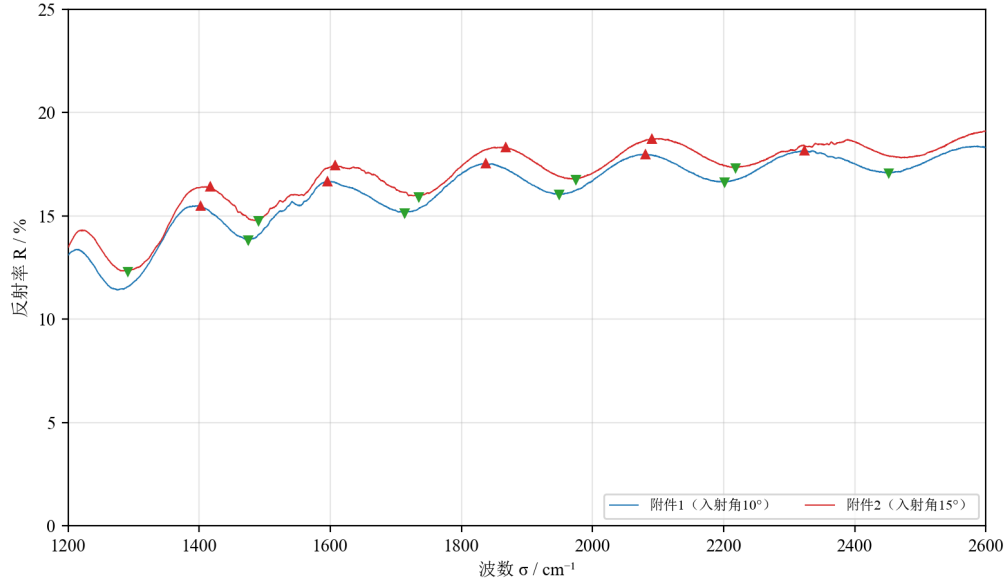


图 6 剔除异常点后的反射光谱及干涉峰谷标记

5.2.3 方案二的求解

按问题一的**方案二**（垂直入射近似），折射率可以直接写出解析解。先由相邻峰谷的反射率 $R_{\max}$ 、 $R_{\min}$ 构造上、下包络线：

$$u(\sigma) = \frac{\sqrt{R_{\max}(\sigma)} + \sqrt{R_{\min}(\sigma)}}{2} \quad (23)$$

$$n(\sigma) = n_0 \frac{1 + u(\sigma)}{1 - u(\sigma)} \quad (24)$$

将式 (23)(24) 应用于处理后的数据，结果列于表 2。由表 2 可见，两份附件所得折射率在数量级与变化趋势上一致，均落在理论范围内，并随波数由 2.22 左右缓慢增大至 2.48 左右。

表 2 通过理论范围筛选的各峰谷对反推折射率

中心波数 σ/cm^{-1}	附件 1 (10°) n	中心波数 σ/cm^{-1}	附件 2 (15°) n
1437.91	2.241	1353.78	2.216
1534.82	2.280	1453.34	2.305
1654.38	2.327	1548.80	2.339
1775.15	2.356	1671.02	2.381
1892.79	2.388	1800.71	2.411
2014.28	2.403	1920.51	2.442
2140.36	2.425	2032.12	2.456

表 2（续）

中心波数 σ/cm^{-1}	附件 1 (10°) n	中心波数 σ/cm^{-1}	附件 2 (15°) n
2261.61	2.432	2153.62	2.477
2386.72	2.447	—	—

表 2 中反推折射率止于约 2.48，未达到理论范围上界 2.60。原因是附件数据在 2500 cm^{-1} 以上的干涉条纹幅度过小，未检出峰谷，无法在更高波数处反推折射率。

相邻峰谷对之间的波数间隔较小，折射率在多数波段变化平缓，因此采用**线性插值**构造连续的 $n(\sigma)$ 曲线，供下一步在峰、谷波数处取值。各对反推值之间不作光滑拟合处理，以减少个别波段少量偏差的影响。筛选后的反推折射率随波数的变化如下图所示。

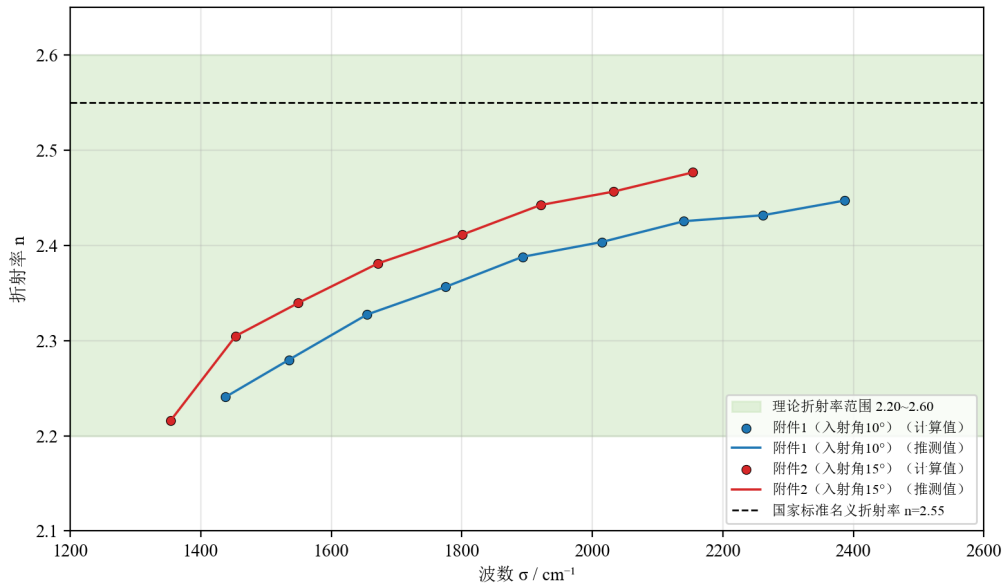


图 7 反推折射率随波数的变化

折射率确定后，由问题一的式 (19) 求解厚度。每一对相邻峰、谷给出厚度的一个估计值：

$$d_j = \frac{1}{4 \left| \sigma_{v,j} \sqrt{n^2(\sigma_{v,j}) - n_0^2 \sin^2 \theta} - \sigma_{p,j} \sqrt{n^2(\sigma_{p,j}) - n_0^2 \sin^2 \theta} \right|} \quad (25)$$

式中 $\sigma_{p,j}$ 、 $\sigma_{v,j}$ 分别为第 j 对峰与谷的波数， $n(\sigma_{p,j})$ 、 $n(\sigma_{v,j})$ 由线性插值在相应波数处取值。

N 对峰谷共得到 N 个厚度估计值 d_j 。由于部分峰谷对反推的折射率可能存在偏差，为减小个别异常值的影响，取中位数作为厚度的最终计算结果，并以中位数绝对偏差 (MAD) 描述各估计值的分散程度：

$$\hat{d} = \text{median}\{d_j\}, \quad \delta d = \text{median}\{|d_j - \hat{d}|\} \quad (26)$$

采用中位数而非平均值，是因为中位数受个别异常值的影响较小。

5.2.4 方案一的求解

接下来按问题一的**方案一**（平均近似）计算。两种偏振的界面反射系数取模为 $|r_s|$ 、 $|r_p|$ ，第二束光振幅为 B ，相位为 ϕ 。将三者代入 5.1 节所得方程，拟合所用模型为：

$$R(\sigma) = \frac{|r_s(\sigma)|^2 + |r_p(\sigma)|^2}{2} + B^2(\sigma) - [|r_s(\sigma)| + |r_p(\sigma)|] B(\sigma) \cos \left(4\pi\sigma d \sqrt{n(\sigma)^2 - n_0^2 \sin^2 \theta} \right) \quad (27)$$

其中 $n(\sigma)$ 由 5.2.3 给出， $B = (\sqrt{R_{\max}} - \sqrt{R_{\min}})/2$ 由 5.1.2 给出， $|r_s|$ 、 $|r_p|$ 由 $n(\sigma)$ 代入 5.1 节的斜入射菲涅耳公式取模得到。因此，上式中唯一待定的量是厚度 d 。以式 (26) 所得 $\hat{d}$ 为初值，在 5.2.2 处理后的有效光谱区内作非线性最小二乘拟合，使模型反射率与实测反射率之差的平方和最小，即可得到 d 的拟合值。

拟合曲线与实测反射率的对比如下图所示，残差曲线绘于右侧纵轴。

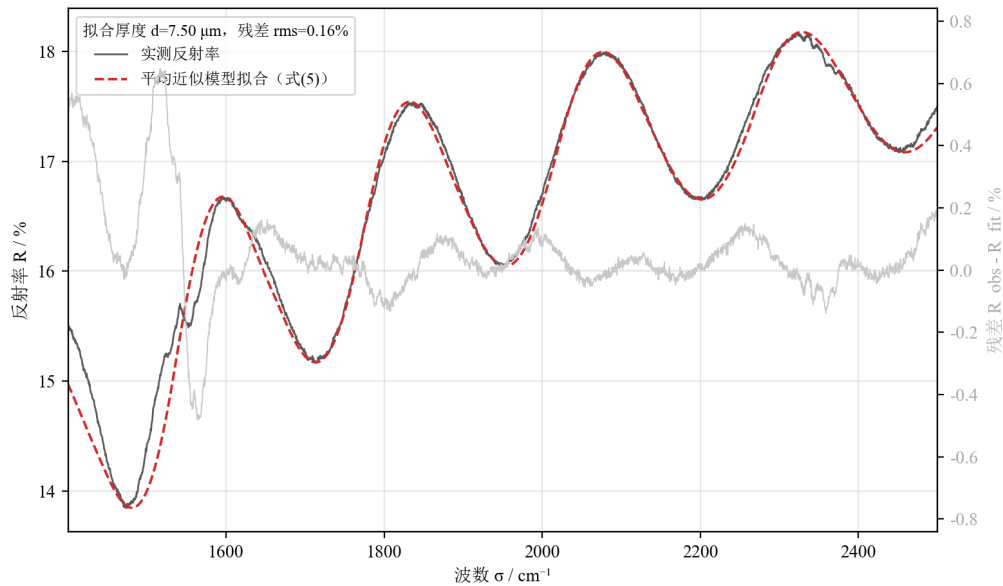


图 8 全谱拟合结果（附件 1，10°）

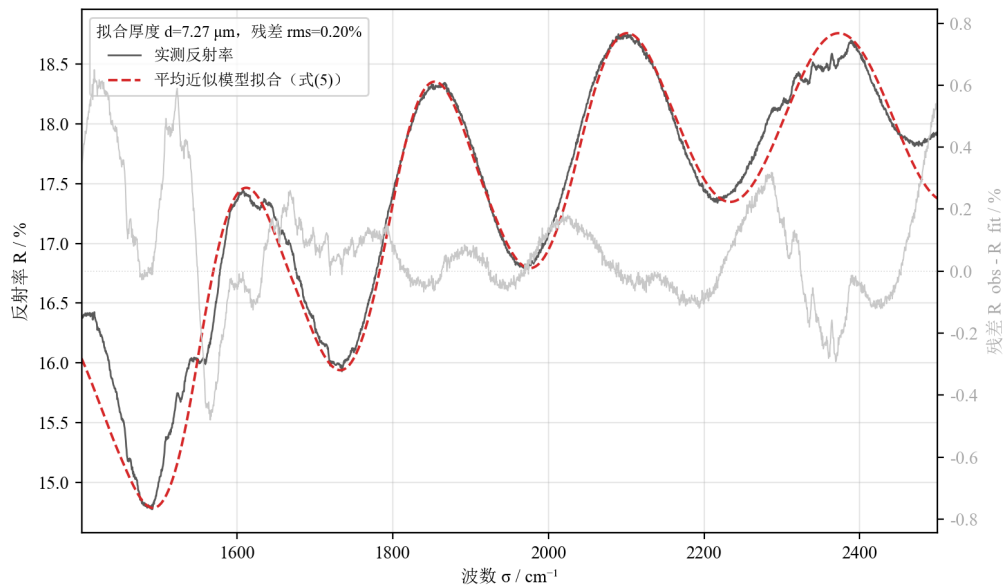


图 9 全谱拟合结果（附件 2，15°）

5.2.5 计算结果

方案二的计算结果：按每一对峰谷计算厚度，结果列于表 3。

表 3 通过理论范围筛选的各峰谷对厚度 d_j (μm)

附件 1 ($\theta = 10^\circ$)					附件 2 ($\theta = 15^\circ$)				
σ_p	σ_v	$n(\sigma_p)$	$n(\sigma_v)$	d_j	σ_p	σ_v	$n(\sigma_p)$	$n(\sigma_v)$	d_j
1401.51	1474.31	2.241	2.255	13.57	1291.11	1416.46	2.216	2.272	7.03
1474.31	1595.32	2.255	2.304	7.15	1416.46	1490.22	2.272	2.318	10.60
1595.32	1713.44	2.304	2.342	7.44	1490.22	1607.38	2.318	2.359	7.43
1713.44	1836.87	2.342	2.373	7.22	1607.38	1734.66	2.359	2.396	6.90
1836.87	1948.72	2.373	2.395	8.12	1734.66	1866.76	2.396	2.428	6.65
1948.72	2079.85	2.395	2.415	7.05	1866.76	1974.27	2.428	2.449	8.31
2079.85	2200.86	2.415	2.428	7.77	1974.27	2089.98	2.449	2.466	7.88
2200.86	2322.36	2.428	2.439	7.84	2089.98	2217.26	2.466	2.477	7.45
2322.36	2451.08	2.439	2.447	7.51	—	—	—	—	—
中位数				7.51	中位数				7.44
MAD				0.32	MAD				0.49

取中位数后得到结果：入射角为 10° 时外延层厚度为 $7.51 \mu\text{m}$ ，入射角为 15° 时为 $7.44 \mu\text{m}$ 。

方案一的计算结果：入射角为 10° 与 15° 时拟合所得厚度分别为 $7.50 \mu\text{m}$ 、 $7.27 \mu\text{m}$ ，残差均方根约 0.16%、0.20%。

两方案的比较：方案二给出厚度 $7.51 \mu\text{m}$ 、 $7.44 \mu\text{m}$ ，方案一给出 $7.50 \mu\text{m}$ 、 $7.27 \mu\text{m}$ ，两方案的差距分别为 0.1%、2.2%，差距较小。方案一并非完全独立，它以方案二的 $\hat{d}$ 为初值作迭代拟合，本质上是对方案二的核对。因此，取方案二两中位数的平均作为外延层厚度 $d \approx 7.48 \mu\text{m}$ 。

5.2.6 结果可靠性分析

首先，反推得到的折射率与文献中的变化规律基本一致。表 2 显示，两组数据的折射率均随波数增大而缓慢增大，数值范围为 2.216 至 2.477，与碳化硅在红外透明区的折射率范围相符，说明所选波段中的干涉条纹具有合理的物理基础。

其次，两种入射角得到的厚度中心值较为接近。按相邻峰谷法计算，入射角为 10° 时的厚度中位数为 $7.51\ \mu\text{m}$ ，入射角为 15° 时为 $7.44\ \mu\text{m}$ ，两者相差约 1.0%。入射角改变后，光在外延层中的传播路径会发生变化，而两组结果仍保持接近，说明厚度计算结果具有较好的稳定性。

两种入射角下各峰谷对的厚度估计如下图所示。表 3 中少数峰谷对得到的厚度与多数结果差别较大，说明峰谷位置和条纹幅度仍受局部噪声、峰谷识别和近似条件的影响。因此，本文采用中位数作为最终结果，以减小少数异常峰谷对计算结果的影响；表中的 MAD 用于说明各峰谷对计算结果的离散程度。

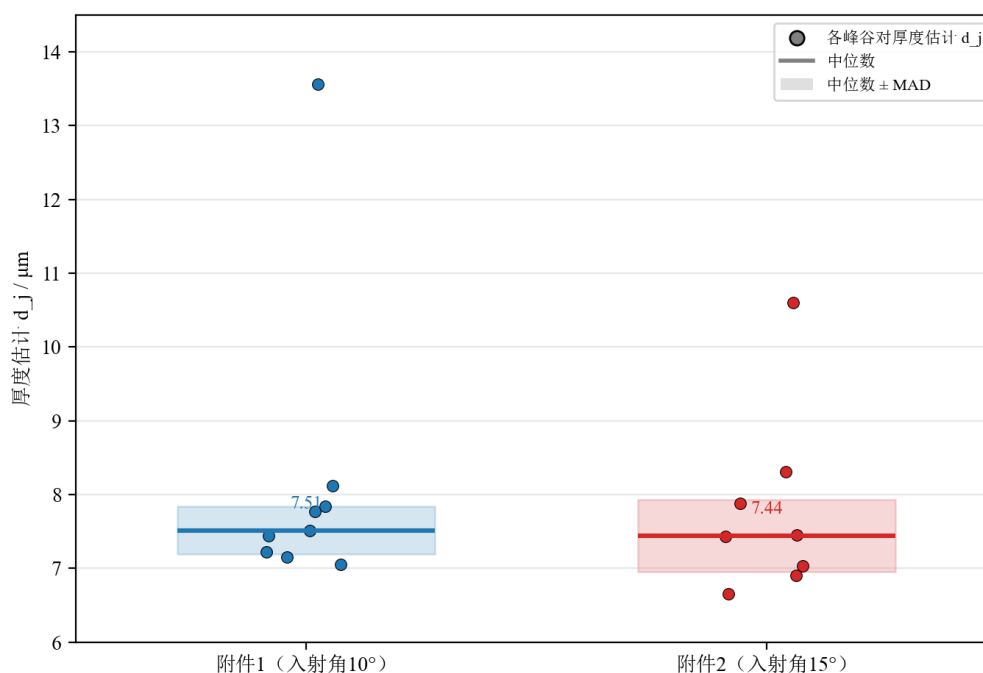


图 10 双入射角厚度估计对比

全谱拟合的结果也给出了补充说明。入射角为 10° 时，全谱拟合厚度为 $7.50\ \mu\text{m}$ ，与相邻峰谷法结果十分接近；入射角为 15° 时，全谱拟合厚度为 $7.27\ \mu\text{m}$ ，与相邻峰谷法相差 $0.17\ \mu\text{m}$ 。两种方法给出的厚度量级一致，同时也反映出折射率反演和两光束近似对结果有一定影响。

综合两种入射角的计算结果，本文取相邻峰谷法两个中位数的平均值，得到碳化硅外延层厚度约为 $d = 7.48\ \mu\text{m}$ 。

5.3 问题三的模型建立与求解

5.3.1 问题分析

题目要求推导产生多光束干涉的必要条件，分析多光束干涉对外延层厚度计算精度可能产生的影响，并确定存在多光束干涉时计算外延层厚度的数学模型。

问题一和问题二中，只保留了两束主要反射光：一束光在外延层上表面直接反射，另一束光进入外延层后，在外延层和衬底的界面反射一次，再从上表面射出。实际测量时，第二束光回到上表面后并不一定全部射出，其中一部分会被反射回外延层。它再次到达下界面后还会发生反射，并重新回到上表面。这样便在原有两束光之外，产生第三束、第四束以及更多反射光。后续各束光的强度逐次衰减。若衰减较慢，各束光的强度仍不可忽略，且彼此保持稳定的相位关系，它们便会与前两束光叠加，形成多光束干涉，使条纹形状发生改变，进而影响折射率与厚度的计算。

后续求解按照图 11 所示的流程进行，先完成理论推导，再分析实测数据。理论推导分两项：一是把光在外延层内多次往返形成的各束反射光逐一写出并求和，得到多光束干涉的反射率表达式，并推导其产生的必要条件；二是将该式与两光束表达式对比，分析条纹形态的偏差，估计多光束干涉对厚度计算精度的影响。理论部分完成后，先对四份附件逐一判别是否出现多光束干涉，再对判定存在多光束干涉的附件 3 与附件 4 给出硅外延层厚度的计算结果，并用独立的算法加以检验。

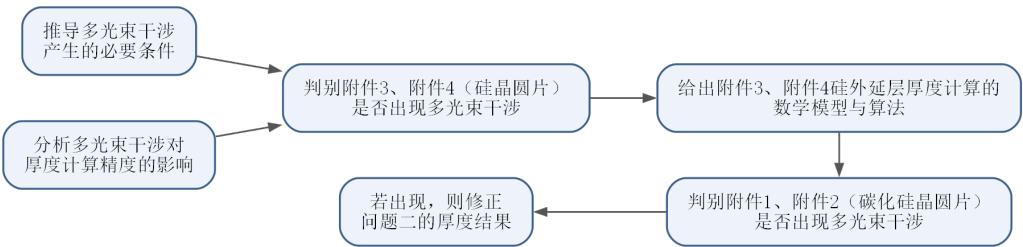


图 11 问题三求解流程图

5.3.2 多光束产生的必要条件

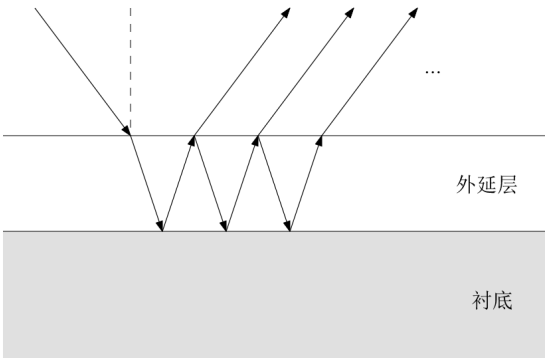


图 12 多光束干涉示意图

如上图所示，与只有一次反射、一次透射的两光束情形不同，光在外延层内的多次往返还会产生第三束及以后的反射光。题目所述的多光束干涉，指后续反射光对实测干涉条纹产生不可忽略影响的情形。后续光束的强弱可以用相邻两束出射光的振幅比衡量。下面从多次反射出发，推导这一振幅比，并给出多光束干涉产生的必要条件。

仍沿用问题一的记号，设入射光的复电场振幅为 $\dot{E}_0$ 。直接在外延层上表面反射的光为：

$$\dot{E}_a = r_{01}\dot{E}_0 \quad (28)$$

进入外延层并在下界面反射一次后射出的光为：

$$\dot{E}_b = B\dot{E}_0 e^{i\phi}, \quad B = t_{01}r_{12}t_{10}A \quad (29)$$

其中， A 表示光在外延层内往返一次后的衰减， ϕ 为这次往返带来的相位差。式 (28) 与式 (29) 就是问题一中的两束反射光。

第二束光回到上表面时，其中一部分会在外延层内侧反射，反射系数为 r_{10} 。这部分光再到达下界面后又会反射一次，并完成一次新的往返。因此，第三束反射光与第二束光相比，额外多乘了系数 $q = r_{10}r_{12}Ae^{i\phi}$ 。同理，第四束光比第三束光再多乘一次 q ，之后各束依次类推。第二束及以后各束光的振幅构成公比为 q 的等比数列。光在每次反射与透射中都有能量损失，公比的模小于 1，该数列收敛。求和后，总反射电场为：

$$\frac{\dot{E}_r}{\dot{E}_0} = r_{01} + \frac{Be^{i\phi}}{1 - r_{10}r_{12}Ae^{i\phi}} \quad (30)$$

相应的反射率为：

$$R_{\text{multi}}(\sigma) = \left| r_{01} + \frac{Be^{i\phi}}{1 - r_{10}r_{12}Ae^{i\phi}} \right|^2, \quad \phi = 4\pi\sigma d\sqrt{n^2(\sigma) - n_0^2 \sin^2 \theta} \quad (31)$$

式 (31) 就是计入多次反射后的反射率表达式。等比数列公比的模决定了后续光束的强弱，记相邻两束出射光的振幅比为：

$$\rho = |r_{10}r_{12}A| \quad (32)$$

ρ 就是该等比数列公比的模，它把界面反射、膜内传播与吸收损耗对后续光束的影响集中在一起。第三束光的振幅是第二束光的 ρ 倍，之后每束再乘一次 ρ ，逐次衰减。若 $\rho \ll 1$ ，后续各束光迅速减弱，多次反射的贡献可以忽略，式 (31) 退化为问题一的两光束模型。只有 ρ 不可忽略时，后续光束才对总反射场有明显贡献，条纹才表现出多光束干涉的特征。因此，界面反射是否较强、吸收损耗是否较小，都体现在 ρ 的大小上。

各束光之间还需保持相干，才能形成稳定的干涉条纹。是否相干取决于相邻两束光

的光程差：光程差不超过光源相干长度时，相位差保持稳定，能够发生干涉；超出这一范围后，相位差随机变化，干涉项会被平均掉。相邻两束光的光程差为：

$$\Delta L = 2nd \cos \theta_1 \quad (33)$$

其中 θ_1 为膜内折射角，符号 $\lesssim$ 表示“小于或近似等于”，即光程差至多接近相干长度而没有明显超出；相干性随光程差增大逐渐退化， L_c 只是量级意义上的特征值，故此处不用严格的小于等于。相干条件为 $\Delta L \lesssim L_c$ ， L_c 为光源的相干长度。

综合以上推导，多光束干涉的产生有两个主要要求。第一，后续反射光不能在少数几次往返后迅速衰减，即式 (32) 的振幅比 ρ 不应过小。第二，参与叠加的各束光之间保持稳定的相位关系，即式 (33) 的光程差 ΔL 不超过光源的相干长度 L_c 。前一条件决定后续光束是否足够强，后一条件决定它们能否稳定叠加。相干长度可由光谱仪的分辨率估算。光源的相干长度约为谱线宽度的倒数，即 $L_c \approx 1/\Delta\sigma$ 。附件数据的波数采样间隔为 $\Delta\sigma \approx 0.48 \text{ cm}^{-1}$ ，分辨率不小于采样间隔，按采样间隔估计得 $L_c \approx 2.1 \times 10^4 \mu\text{m}$ 。取 5.3.5 所得厚度 $d \approx 3.4 \mu\text{m}$ 代入式 (33)，相邻光束的光程差约为 $23 \mu\text{m}$ ，只有相干长度的千分之一。即使按常规测量常用的 4 cm^{-1} 分辨率估计，相干长度仍比光程差大两个量级以上。因此，相干条件充分满足，相邻光束能够稳定叠加。

由于附件仅提供波数与反射率数据，各界面的反射系数与膜内传播损耗无法直接测得。下面利用两光束模型从实测峰谷反射率估计 ρ ，并结合条纹形态指标进行判别，见 5.3.3。

5.3.3 多光束干涉对厚度计算精度的影响分析

式 (31) 与问题一、问题二的两光束表达式相比，结构上的差别是多出一个分母因子，多光束干涉对厚度计算的影响即来源于此。本节先分析该因子给条纹形态与峰谷反射率读数带来的改变，再估计折射率与厚度的偏差，最后依据实测条纹的形态给出判别多光束干涉的判据。

两光束情形的反射率为 $R = |r_{01} + Be^{i\phi}|^2$ ，即问题一的式 (5)。 ρ 很小时，式 (31) 的分母接近 1，退化为式 (5)，两光束模型是式 (31) 的最低阶近似。 ρ 不可忽略时，分母随相位周期性变化，相当于把第二束及后续各束光的振幅作周期性放大。多光束干涉对厚度计算的影响，全部来自这一分母。

分母的作用最直接地体现在条纹形态上。分母的模在干涉谷对应的相位附近最小，各束光在该处叠加得更充分，谷变得更尖、更深；在干涉峰对应的相位附近分母最大，峰则变得更宽、更平缓。条纹的调制深度也随之改变，相邻峰谷之间大部分数据点会偏向中线一侧。

为直观展示两种条纹的区别，按硅的折射率与实测峰谷反射率确定式 (29)、式 (31)

中的各系数，数值合成两种条纹，如下图所示。图中 $|r_{01}| = 0.548$ ，硅的折射率取 3.42， B 由实测峰谷反射率 $R_{\max} \approx 39\%$ 、 $R_{\min} \approx 21\%$ 确定。为清晰展示形态差异，合成时 ρ 取 0.20，大于硅晶圆的实际值 0.05，两条曲线的差异方向与实际相同。

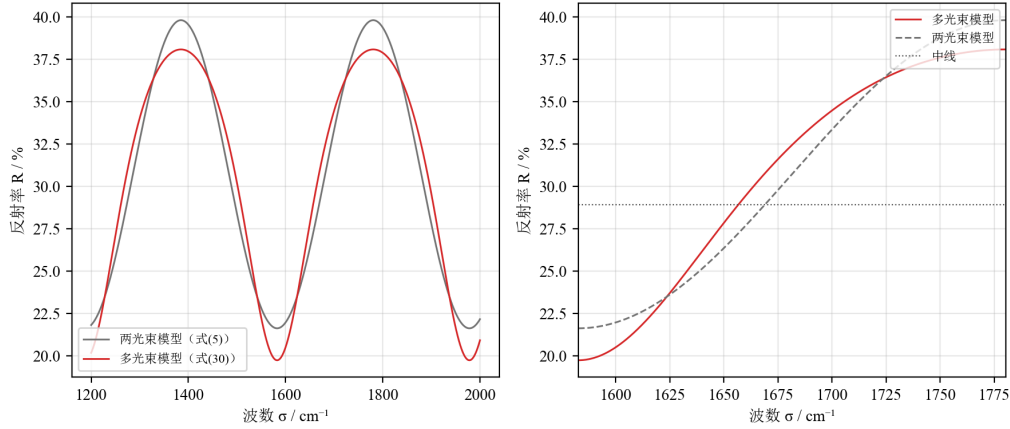


图 13 两光束与多光束干涉条纹对比

由图可见，两光束曲线是标准的余弦形条纹；多光束曲线的谷明显变尖，峰明显变宽，相邻峰谷之间大部分数据点位于中线上方。中线指该峰谷区间内峰、谷反射率的平均值 $R_{\text{mid}} = (R_{\max} + R_{\min})/2$ 。一个峰谷区间内位于中线以下的点占比，两光束恰为 50%，多光束则明显更小。上图中该差异经过放大，实测硅光谱的偏离幅度较小，但偏离方向一致。因此，这一占比可以作为定量判别的依据。

因此，判别采用两个互补的判据，一个估计相邻光束的振幅比，一个检验条纹形态。振幅比可用两光束模型反推。第二束反射光的振幅 B 由相邻峰、谷的反射率确定，即问题一的式 (6)。把它与式 (29) 对比，即可反推衬底界面的反射系数，进而得到振幅比的估计值：

$$\hat{\rho} = |r_{10}| \hat{r}_{12}, \quad \hat{r}_{12} = \frac{B}{t_{01}t_{10}A} \approx \frac{\sqrt{R_{\max}} - \sqrt{R_{\min}}}{2t_{01}t_{10}} \quad (34)$$

其中 r_{10} 、 t_{01} 、 t_{10} 按垂直入射近似，由外延层折射率的文献名义值算出；外延层在测量波段内吸收较弱，取 $A \approx 1$ 。两个阈值由数值模拟确定：按硅的界面参数合成含分母因子的反射谱，逐次改变振幅比，考察峰谷反射率读数与反推折射率的偏差何时达到不可忽略的水平。若 $\hat{\rho} \lesssim 0.03$ ，第三束光可以忽略，两光束模型足够；若 $\hat{\rho} \gtrsim 0.05$ ，则应采用多光束模型。

振幅比的估计只用到峰谷反射率，属于量级判断。条纹形态是多光束干涉更直接的证据，可作独立的检验。两光束干涉的反射率随相位按余弦变化，相邻峰、谷之间位于中线以下的数据点应约占一半。多光束干涉谷变尖、峰变宽，该占比偏离 50%。记第 j

对相邻峰、谷之间位于中线以下的数据点占比为 η_j ，对全部峰谷对取平均：

$$\eta = \frac{1}{J} \sum_{j=1}^J \eta_j \quad (35)$$

η 的正常波动范围由数值模拟确定。两光束条纹严格呈余弦形， η 应恰为 50%；在合成的两光束谱上叠加与实测水平相当的随机扰动后重复统计，得到的波动范围约为 $50\% \pm 2\%$ ，代表测量扰动单独造成的偏离。若 η 落在该范围之内，条纹可视为余弦形，两光束模型成立；若显著偏离，则条纹已带有多光束干涉的形态特征。

5.3.4 判别多光束干涉求解

按 5.3.3 的判据对四份附件逐一统计。峰、谷检测的显著度门槛与 5.2 节相同，反射率为零或负的点先行剔除。附件 1、2 的统计中剔除了两光束模型失效的残余射线带 $787 \sim 968 \text{ cm}^{-1}$ 。每个峰谷区间接式 (34) 给出 $\hat{\rho}$ 的逐对估计并取中位数，按式 (35) 统计 η ，结果列于表 4。

表 4 四份附件的多光束干涉定量判别结果

附件	峰谷段数	低于中线占比 η	振幅比 $\hat{\rho}$	判别结论
附件 1 (SiC, 10°)	23	50.5%	0.013	两光束模型适用
附件 2 (SiC, 15°)	21	51.2%	0.015	两光束模型适用
附件 3 (Si, 10°)	15	43.2%	0.047	存在多光束干涉
附件 4 (Si, 15°)	14	43.8%	0.040	存在多光束干涉

碳化硅晶圆片两份附件的 η 均落在 $50\% \pm 2\%$ 之内， $\hat{\rho}$ 也小于 0.03，条纹呈余弦形，两光束模型足够精确，问题二的厚度结果不受多光束干涉的明显影响。硅晶圆片两份附件的 η 均在 43% 左右，显著低于 50%， $\hat{\rho}$ 已接近 0.05，峰形指标与振幅比两种证据相互印证，且两种入射角下的判断一致。因此判定附件 3 与附件 4 的测试结果中出现了多光束干涉。

判定存在多光束干涉后，反射率表达式由两光束的式 (5) 变为式 (31)。分母因子在干涉谷附近使谷变尖变深，在干涉峰附近使峰变宽变缓，实测峰谷反射率不再是问题一式 (6) 所给出的包络值，式 (8) 的振幅分解也随之失效。因此，包络线法不能再用于反演折射率，折射率应取文献值。厚度的计算见 5.3.5。

5.3.5 硅外延层厚度的计算

5.3.4 已判定附件 3、4 存在多光束干涉，包络线法不能再用于反演折射率，折射率取硅的文献名义值 $n = 3.42$ 。数据预处理与 5.2 节相同，删除反射率为零或负的点。先按问题一的式 (19)，以常数折射率对全部峰谷对逐对求厚度。其中，波数低于 1500 cm^{-1} 的峰谷对给出的厚度约为 $3.8 \sim 3.9 \mu\text{m}$ ，比高波数段的 $3.4 \mu\text{m}$ 偏大 $10\% \sim 15\%$ ，其余各对则彼此接近。由此可见，名义折射率只适用于高波数段；重掺杂衬底在低波数端的折射

率偏离名义值较多，并伴随较强的吸收，常数折射率已不能描述全谱。为把低波数段也纳入计算，方案一把衬底的色散写成式 (36) 的 Drude 介电函数，对全谱段作拟合；方案二沿用相邻峰谷法，补充界面反射相移的色散修正；统计仍不计入波数低于 1500 cm^{-1} 的区段，该区段的谷位于谱段边缘附近，识别不完整。重掺杂衬底的介电函数取 Drude 形式（波数以 cm^{-1} 计）：

$$\tilde{\epsilon}_2(\sigma) = \epsilon_\infty - \frac{\sigma_p^2}{\sigma^2 + i\gamma\sigma} \quad (36)$$

其中 σ_p 为等离子体波数， γ 为阻尼波数。衬底与外延层同为硅，高频介电常数取 $\epsilon_\infty = n^2 = 11.7$ 。把式 (36) 代入菲涅耳公式，得到衬底界面的复反射系数 $r_{12}(\sigma)$ ，其模与幅角都随波数变化；再代入式 (31)，即得以 d 、 σ_p 、 γ 为参数的全谱反射率模型，在全区间 $400 \sim 4000\text{ cm}^{-1}$ 上作全局非线性最小二乘拟合。厚度初值由傅里叶基频锚定，以免拟合被低波数端的局部极小带偏；衬底参数先在多组初值上网格搜索，再全参数精修。附件 3 的拟合厚度为 $3.40\text{ }\mu\text{m}$ ，等离子体波数 3594 cm^{-1} ，阻尼波数 426 cm^{-1} ，残差均方根 2.60% ；附件 4 的拟合厚度为 $3.37\text{ }\mu\text{m}$ ，等离子体波数 3675 cm^{-1} ，阻尼波数 354 cm^{-1} ，残差均方根 1.32% 。拟合曲线见图 14。拟合覆盖了含低波数端在内的全谱段，残差主要集中在低波数端的深谷附近，这是用单一 Drude 项描述衬底的简化所致。拟合参数对应的屏蔽等离子体边缘 $\sigma_p/\sqrt{\epsilon_\infty}$ 约为 1050 cm^{-1} ；取电导有效质量 $m^* \approx 0.26m_e$ ，折算衬底掺杂浓度约 $4 \times 10^{19}\text{ cm}^{-3}$ ，与重掺杂衬底的量级一致。

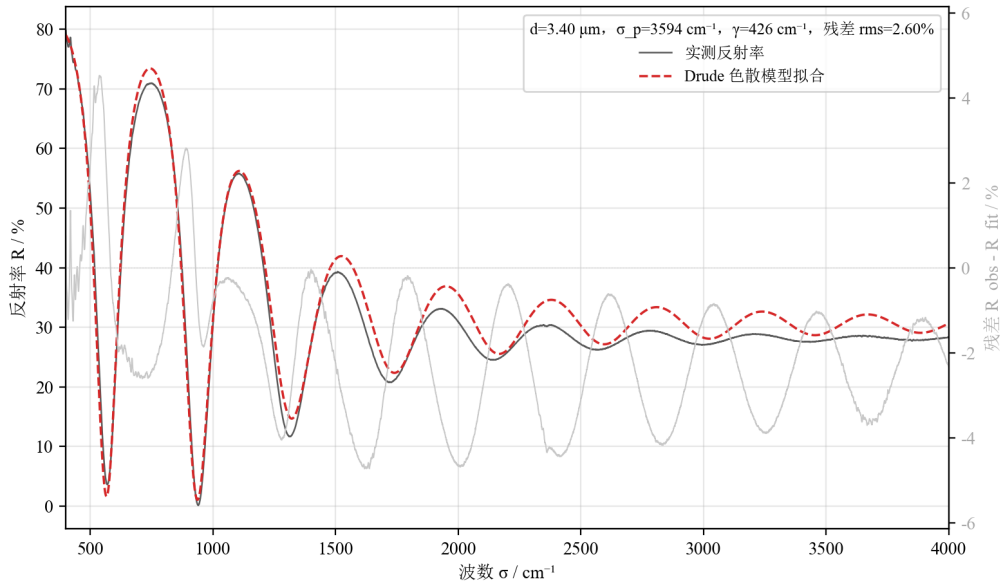


图 14 方案一拟合结果（附件 3， 10° ）

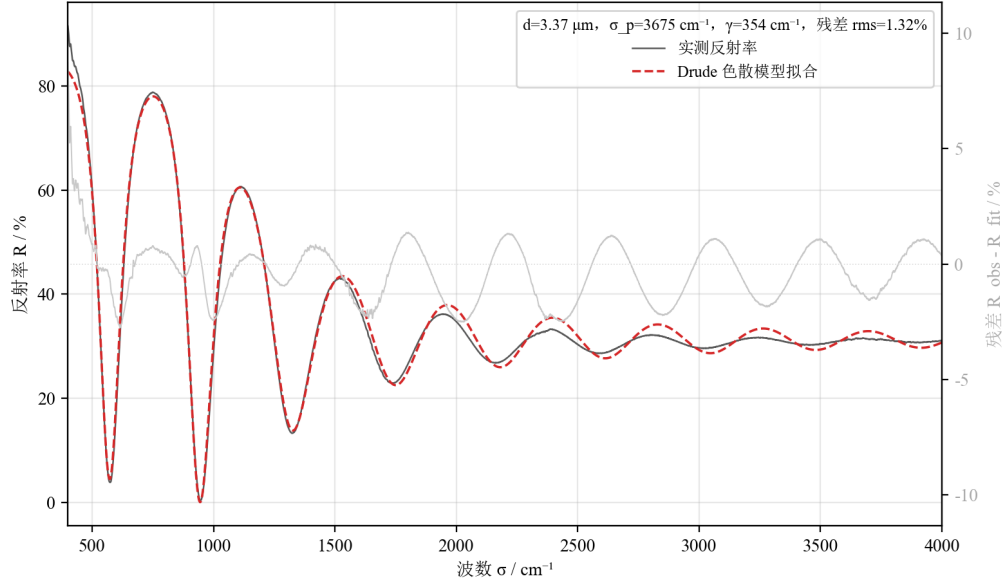


图 14 方案一拟合结果（附件 4，15°）

方案二从峰谷位置出发。两光束模型的峰谷位置仅由 $\phi = k\pi$ 决定；在式 (31) 中，分子与分母的模都只依赖同一组合 $\cos(\phi + \delta_{12})$ ，反射率随该组合单调变化，因此条纹的极值位于：

$$\phi(\sigma) + \delta_{12}(\sigma) = k\pi, \quad \delta_{12} = \arg r_{12} \quad (37)$$

δ_{12} 不随波数变化时，式 (37) 退化为两光束的峰谷条件，即问题一的式 (19)。存在衬底色散时， δ_{12} 随波数明显变化，仍按常数相移处理会带来系统偏差。相邻峰、谷的相位差仍为 π ，逐对解出厚度：

$$d_j = \frac{\pi - \delta_{12}(\sigma_{2,j}) + \delta_{12}(\sigma_{1,j})}{4\pi c (\sigma_{2,j} - \sigma_{1,j})}, \quad c = \sqrt{n^2 - n_0^2 \sin^2 \theta} \quad (38)$$

$\delta_{12}(\sigma)$ 由方案一拟合的 Drude 参数算出。式 (38) 的分子含有两个波数处的相移之差，相移随波数的变化已包含在内。逐对代入的结果列于表 5，未经相移修正的式 (19) 结果一并列出以便对照。各对估计值取中位数作为厚度结果，以 MAD 描述分散程度，统计方式同 5.2.3。

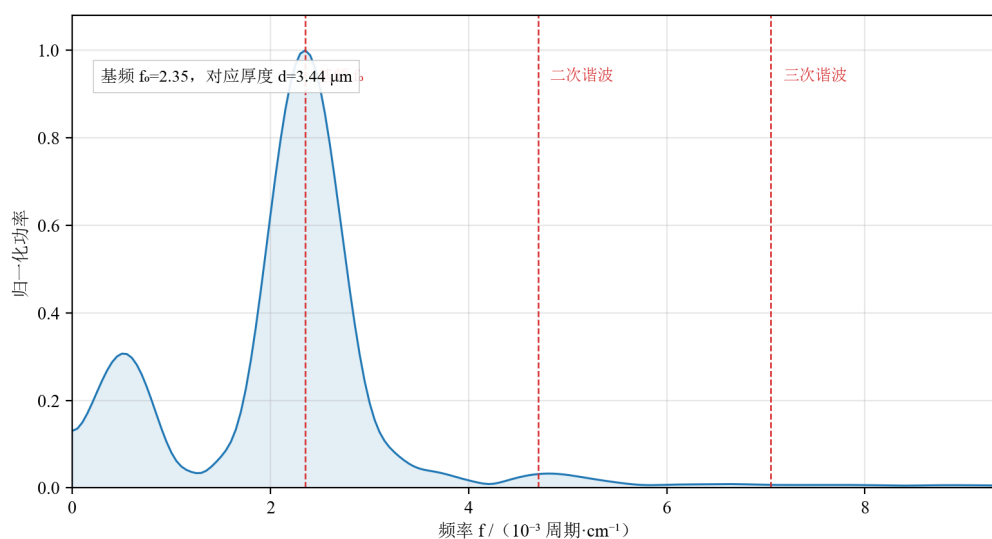
表 5 附件 3、4 各峰谷对的厚度 d_j (μm)

附件 3 ($\theta = 10^\circ$)				附件 4 ($\theta = 15^\circ$)			
σ_1	σ_2	d_j 未修正	d_j 修正	σ_1	σ_2	d_j 未修正	d_j 修正
1508.06	1722.60	3.41	3.34	1519.15	1736.10	3.38	3.31
1722.60	1927.02	3.58	3.53	1736.10	1941.97	3.56	3.52
1927.02	2141.56	3.41	3.38	1941.97	2160.85	3.35	3.32
2141.56	2347.43	3.56	3.53	2160.85	2387.44	3.24	3.21
2347.43	2564.86	3.37	3.35	2387.44	2587.04	3.67	3.66
2564.86	2779.89	3.40	3.39	2587.04	2802.54	3.40	3.39

表 5 (续)

附件 3 ($\theta = 10^\circ$)				附件 4 ($\theta = 15^\circ$)			
σ_1	σ_2	d_j 未修正	d_j 修正	σ_1	σ_2	d_j 未修正	d_j 修正
2779.89	2993.46	3.43	3.41	2802.54	3009.37	3.54	3.53
2993.46	3203.66	3.48	3.47	3009.37	3245.13	3.11	3.10
3203.66	3434.60	3.17	3.16	3245.13	3450.03	3.58	3.57
中位数		3.41	3.39	中位数		3.40	3.39
MAD		0.05	0.05	MAD		0.16	0.15

方案二的结果还可以用傅里叶变换独立检验。干涉条纹相当于反射率随波数作周期振荡，基频只由厚度决定；多光束干涉只会带来整倍频的谐波，不会改变基频。对波数不低于 1500 cm^{-1} 的谱段，先插值到均匀的波数间隔，扣除缓慢变化的趋势后作傅里叶变换，由功率谱的主峰读出基频。图 15 给出两份附件的功率谱，主峰尖锐突出，二次、三次谐波清晰可辨，与 5.3.4 的判别结论一致。由基频算得的厚度为 $3.44 \mu\text{m}$ 与 $3.45 \mu\text{m}$ ，比未修正的峰谷中位数偏高约 1%。偏高的原因是傅里叶基频把条纹相位中的相移色散一并计入；频率读数的最小间隔约合厚度 $0.07 \mu\text{m}$ ，也限制了两者的比对精度。相移修正后，两份附件的峰谷中位数均为 $3.39 \mu\text{m}$ ，与方案一的拟合厚度相符，三种方法在不确定度范围内相互印证。

图 15 反射谱的傅里叶功率谱（附件 3， 10° ）

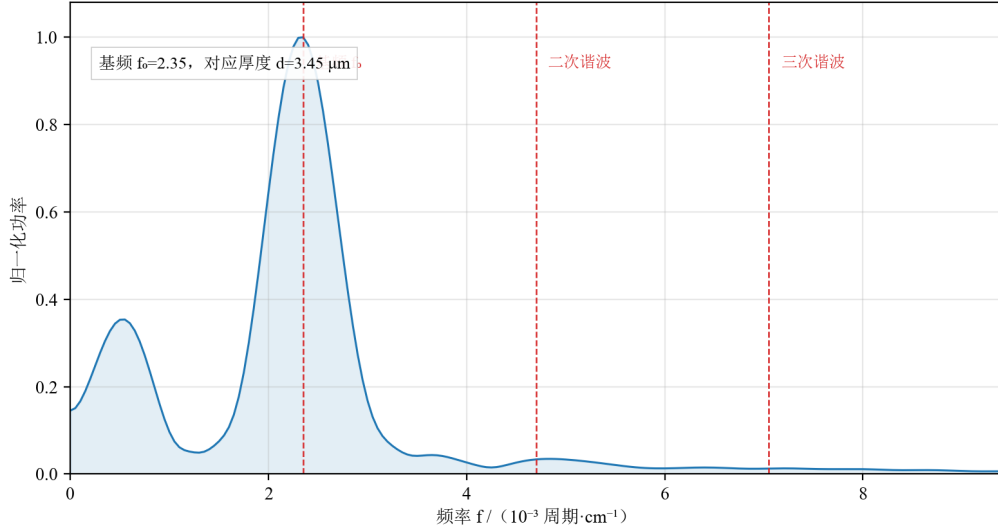


图 15 反射谱的傅里叶功率谱（附件 4，15°）

最后分析结果的可靠性。第一，两种入射角得到的厚度彼此接近。入射角改变后，光在外延层中的传播路径随之变化，两组结果仍保持一致，说明厚度计算具有较好的稳定性。第二，各峰谷对估计值的分散程度有限，附件 3 的 MAD 约为中位数的 1%，附件 4 约为 4%，与峰谷位置的识别误差水平相当。第三，方案一利用全谱线形，方案二利用峰谷位置，傅里叶检验利用频域结构，三者依据相互独立，结果彼此一致。综合各结果，硅外延层厚度约为 $d = 3.39 \mu\text{m}$ 。

5.3.6 总结

本节按题目要求逐项给出问题三的回答。多光束干涉产生的必要条件有两个。**第一，式 (32) 定义的相邻出射光束振幅比 ρ 不可过小**，否则后续光束迅速衰减，式 (31) 退化为两光束模型。**第二，式 (33) 的相邻光束光程差 ΔL 应不超过光源的相干长度 L_c** ；按采样间隔估计，光程差只有相干长度的千分之一，该条件充分满足。

多光束干涉对厚度计算精度的影响体现在两处。**第一，式 (31) 多出的分母因子使干涉谷变尖、峰变宽**，实测峰谷反射率不再是两光束模型的包络值，包络线法反演折射率随之失效，折射率只能取文献名义值。**第二，重掺杂衬底的界面反射相移随波数变化**，峰谷位置偏离两光束的极值条件，若不作修正，峰谷法会残留约百分之一量级的系统偏差。

按 5.3.3 的振幅比估计与中线占比两项判据对四份附件逐一判别。附件 3、4 的低于中线占比 η 约为 43%，振幅比估计值 $\hat{\rho}$ 约为 0.04 ~ 0.05，频谱中又见整倍频谐波，三种证据一致，判定**两份硅晶圆片的测试结果中出现了多光束干涉**。附件 1、2 的 η 均落在 $50\% \pm 2\%$ 之内， $\hat{\rho}$ 小于 0.03，条纹呈余弦形，两光束模型足够精确，不存在需要消除的多光束影响，**问题二的碳化硅厚度结果不受影响**。

针对存在多光束干涉的硅晶圆片，厚度计算以式 (31) 为基本模型，采用两种算法：方案一把衬底色散写成式 (36) 的 Drude 介电函数，对全谱段作非线性最小二乘拟合；方案二按式 (38) 逐对求解计入相移色散修正的相邻峰谷厚度。两种入射角下，方案一给出 $3.40\ \mu\text{m}$ 与 $3.37\ \mu\text{m}$ ，方案二均给出 $3.39\ \mu\text{m}$ ，傅里叶检验给出 $3.44\ \mu\text{m}$ 与 $3.45\ \mu\text{m}$ 。综合各结果，**硅外延层厚度约为 $d = 3.39\ \mu\text{m}$ 。**

六、模型评价

6.1 模型的优点

1. 两光束模型通过条纹幅度与条纹位置求解折射率与厚度信息，两条求解路径分离，每一步都可以单独检验。
2. 三个问题在同一框架下递进，两光束情形是多光束反射率式 (31) 的最低阶近似，多光束判别与厚度计算又各有多种独立证据相互印证。

6.2 模型的缺点

1. 偏振按算术平均近似处理，入射角较大时存在系统偏差。
2. 峰谷法对噪声敏感，个别峰谷对的厚度偏差可达一成以上；且常数折射率在低波数端不再适用。
3. 多光束干涉下折射率只能取文献名义值，判别阈值与相干长度也依赖数值模拟与采样估算，换用其他材料与仪器时需重新标定。

6.3 模型的改进方向

1. 用计入两种偏振分量的严格反射率模型替代算术平均近似，减弱大入射角下的近似误差。
2. 低波数段改用更精细的色散模型描述衬底折射率与吸收随波数的变化，提高深谷处的拟合精度，并使该谱段重新参与厚度统计。
3. 在多光束影响可以忽略的波段，用包络线法反演折射率，与文献名义值交叉检验。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色，详细使用情况见支撑材料。

参考文献

- [1] 刘朝霞, 楚合营, 黄新成, 张景川. 反射光谱包络线法测量光电薄膜的光学常数和厚度[J]. 大学物理实验, 2015, 28(3): 11-14.
- [2] PITMAN K M, HOFMEISTER A M, CORMAN A B, SPECK A K. Optical properties of silicon carbide for astrophysical applications I. New laboratory infrared reflectance spectra and optical constants[J]. Astronomy & Astrophysics, 2008, 483(2): 661-672.
- [3] WANG S, ZHAN M, WANG G, et al. 4H-SiC: a new nonlinear material for midinfrared lasers[J]. Laser & Photonics Reviews, 2013, 7(5): 831-838.

附录

支持文件清单

 5.1画图.py
 5.2画图.py
 5.2可靠性核查.py
 5.2求解.py
 5.3Drude求解.py
 5.3厚度求解.py
 5.3画图.py
 5.3检验.py
 5.3判别.py
 5.3判别模拟.py
 5.3求解.py
 AI 工具使用详情.pdf
 问题二算法流程图.dot
 问题三求解流程图.dot

程序代码

5.1 画图.py

```
# -*- coding: utf-8 -*-
"""
5.1 画图：5.1 节全部插图
=====

图2 干涉条纹与相邻峰谷相位差示意图（5.1.3，说明峰谷位置携带厚度信息）
图3 双偏振反射系数模值随入射角的变化（5.1.2，支撑方案二的垂直入射近似）
图9 包络线法示意图（5.1.2，说明包络线消去光 2 的贡献、留下界面反射振幅 u）

模型（问题一式(5)，垂直入射近似下的示意形式）：

$$R(\sigma) = |r_{01}|^2 + B^2 + 2|r_{01}| B \cos \varphi, \quad \varphi = 4\pi\sigma d \cdot \sqrt{(n^2 - n_0^2 \sin^2 \theta)}$$

相邻波峰与波谷对应  $\cos \varphi = +1$  与  $-1$ ，相位差  $\Delta \varphi = \pi$ ，其波数间隔  $\Delta \sigma = 1/(4d\sqrt{(n^2 - \sin^2 \theta)})$ 。

用法：python "5.1画图.py"（输出至 .././ 图片/ 目录）
运行环境：Python 3 + numpy / matplotlib
字体：中文 SimSun（宋体），西文与数学符号 Times New Roman
"""

from pathlib import Path
import numpy as np
import matplotlib
matplotlib.use('Agg')
```

```

import matplotlib.pyplot as plt

plt.rcParams['font.family'] = ['Times New Roman', 'SimSun']
plt.rcParams['mathtext.fontset'] = 'stix'
plt.rcParams['axes.unicode_minus'] = False

# ----- 示意参数（取碳化硅的典型值） -----
N0 = 1.0          # 空气折射率
N_SIC = 2.40      # 外延层折射率（示意取常数，忽略色散）
D_UM = 7.30      # 外延层厚度  $\mu\text{m}$ 
THETA_DEG = 10.0 # 入射角（度）
B_AMP = 0.05     # 光 2 振幅系数（远小于光 1，两光束近似成立）
SIGMA_LO, SIGMA_HI = 1400.0, 2500.0 # 波数范围  $\text{cm}^{-1}$ 

OUT_DIR = Path(__file__).resolve().parent.parent.parent / '图片'

# 标注文字的白底衬底：即使压到曲线上也能清晰阅读
WHITE_BOX = dict(boxstyle='round,pad=0.3', facecolor='white',
                  edgecolor='none', alpha=0.85)

def phase_factor():
    """返回  $K = n \cdot \cos\theta_1 = \sqrt{(n^2 - n_0^2 \sin^2\theta)}$ 。"""
    th = np.radians(THETA_DEG)
    return np.sqrt(N_SIC ** 2 - (N0 * np.sin(th)) ** 2)

def reflectance(sigma_cm):
    """两光束模型的反射率  $R(\sigma)$ （返回百分数）。"""
    d_cm = D_UM * 1e-4
    r01 = (N_SIC - N0) / (N_SIC + N0) # 垂直入射菲涅耳系数
    phi = 4 * np.pi * sigma_cm * d_cm * phase_factor()
    R = r01 ** 2 + B_AMP ** 2 + 2 * r01 * B_AMP * np.cos(phi)
    return R * 100.

def peak_valley_sigma(m=6):
    """第 m 个波峰与其后相邻波谷的波数（ $\text{cm}^{-1}$ ）。"""
    峰:  $\varphi = 2m\pi \rightarrow \sigma = m/(2dK)$ 
    谷:  $\varphi = (2m+1)\pi \rightarrow \sigma = (2m+1)/(4dK)$ 
    """
    d_cm, K = D_UM * 1e-4, phase_factor()
    sigma_p = m / (2 * d_cm * K)
    sigma_v = (2 * m + 1) / (4 * d_cm * K)
    return sigma_p, sigma_v

def main():
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    sg = np.linspace(SIGMA_LO, SIGMA_HI, 2000)
    Rv = reflectance(sg)
    s_p, s_v = peak_valley_sigma()

    fig, ax = plt.subplots(figsize=(8, 4.8))
    ax.plot(sg, Rv, lw=1.4, color='#1f4e79', zorder=2)

```

```

# 相邻峰、谷标记（用图例说明，不再单独加指示箭头与文字）
ax.plot([s_p], [reflectance(s_p)], '^', ms=9, color='tab:red',
        label='相邻波峰', zorder=4)
ax.plot([s_v], [reflectance(s_v)], 'v', ms=9, color='tab:green',
        label='相邻波谷', zorder=4)

# 峰谷之间的相位差标注：整条标注线抬到曲线最高点之上，留出明显间隙避免与曲线重叠
rng = Rv.max() - Rv.min()
y_arrow = Rv.max() + rng * 0.34
# 标注线两端向下引虚线，指明它对应的是哪两个波数
ax.vlines([s_p, s_v], [reflectance(s_p), reflectance(s_v)], y_arrow,
          color='0.55', ls=':', lw=1.0, zorder=3)
ax.annotate('', xy=(s_p, y_arrow), xytext=(s_v, y_arrow),
            arrowprops=dict(arrowstyle='<->', color='#333333', lw=1.2),
            zorder=5)
ax.text((s_p + s_v) / 2, y_arrow + rng * 0.05,
        '相邻峰谷相位差  $\Delta\phi = \pi$ ', ha='center', va='bottom', fontsize=12,
        bbox=WHITE_BOX)

ax.text((s_p + s_v) / 2, Rv.min() - rng * 0.20,
        r'$\Delta\sigma = \dfrac{1}{4d\sqrt{n^2-n_0^2\sin^2\theta}}$'
        f'  $\approx \{s_v - s_p\} f$ ' + r'$\mathrm{cm}^{-1}$',
        ha='center', fontsize=11, bbox=WHITE_BOX)

ax.set_xlabel('波数  $\sigma / \mathrm{cm}^{-1}$ ')
ax.set_ylabel('反射率 R / %')
ax.set_xlim(SIGMA_LO, SIGMA_HI)
ax.set_ylim(Rv.min() - rng * 0.40, Rv.max() + rng * 0.72)
ax.grid(alpha=0.3)
ax.legend(fontsize=9, loc='upper right')
fig.tight_layout()

name = '图2 干涉条纹与相邻峰谷相位差示意图'
fig.savefig(OUT_DIR / f'{name}.svg', dpi=300)
fig.savefig(OUT_DIR / f'{name}.png', dpi=200)
plt.close(fig)
print(f'已保存 {OUT_DIR / name}.svg')
print(f'峰  $\sigma_p = \{s_p\} \mathrm{cm}^{-1}$ , 谷  $\sigma_v = \{s_v\} \mathrm{cm}^{-1}$ , '
      f'间隔  $\Delta\sigma = \{s_v - s_p\} \mathrm{cm}^{-1}$ ')

```

----- 图 3

def plot_polarization():

"""图3：双偏振反射系数模值随入射角的变化（对应问题一式(10)~(14)）。

取 $n \approx 2.4$ 画出 $|r_s(\theta)|$ 、 $|r_p(\theta)|$ 与垂直入射值 $|r_0|$ ；再画出按光强平均后的等效反射振幅 $\sqrt{(|r_s|^2 + |r_p|^2)/2}$ 。可见单独看某种偏振， 10° 、 15° 处的偏差约 1%~3%；但两种偏振按光强平均后，等效振幅几乎不随角度变化，与垂直入射值重合，这正是方案二（垂直入射近似）可以成立的依据。

"""

```

th = np.linspace(0.0, 30.0, 601)
c = np.cos(np.radians(th))
sin1 = N0 * np.sin(np.radians(th)) / N_SIC
cos1 = np.sqrt(1.0 - sin1 ** 2)
rs = np.abs((N0 * c - N_SIC * cos1) / (N0 * c + N_SIC * cos1))

```

```

rp = np.abs((N_SIC * c - N0 * cos1) / (N_SIC * c + N0 * cos1))
r_unpol = np.sqrt((rs ** 2 + rp ** 2) / 2.0)      # 光强平均的等效反射振幅
r0 = (N_SIC - N0) / (N_SIC + N0)                 # 垂直入射值

fig, ax = plt.subplots(figsize=(8, 4.8))
ax.plot(th, rs, lw=1.6, color='#1f77b4', label='s 偏振 |rs(θ)|')
ax.plot(th, rp, lw=1.6, color='#d62728', label='p 偏振 |rp(θ)|')
ax.plot(th, r_unpol, lw=2.0, ls='--', color='#2ca02c',
        label='光强平均等效振幅  $\sqrt{(rs^2+rp^2)/2}$ ')
ax.axhline(r0, color='k', lw=1.2, ls='--',
        label=f'垂直入射 |r0| = {r0:.4f}')
# 标出本题的两个工作角
for ang, col in ((10.0, '#1f77b4'), (15.0, '#d62728')):
    ax.axvline(ang, color='0.6', ls=':', lw=1.0)
    ax.text(ang + 0.4, r0 + 0.011, f'{ang:.0f}°', fontsize=11, color='0.35')
ax.set_xlabel('入射角 θ / °')
ax.set_ylabel('反射系数模值')
ax.set_xlim(0, 30)
ax.legend(fontsize=9, loc='center left')
ax.grid(alpha=0.3)

name = '图3 双偏振反射系数模值随入射角的变化'
fig.tight_layout()
fig.savefig(OUT_DIR / f'{name}.svg', dpi=300)
fig.savefig(OUT_DIR / f'{name}.png', dpi=200)
plt.close(fig)
print(f'已保存 {OUT_DIR / name}.svg')

```

----- 图 9

def plot_envelope_demo():

"""图9: 包络线法示意图 (对应问题一式(5)-(8))。"""

上图: 两光束干涉的振荡谱 $R(\sigma)$ 与上、下包络线 $R_{\max}$ 、 $R_{\min}$;

下图: 把上、下包络开方, 画出 $\sqrt{R_{\max}}$ 、 $\sqrt{R_{\min}}$ 及二者的中线 $u=(\sqrt{R_{\max}}+\sqrt{R_{\min}})/2$ 。

开方之后两束光的振幅变成一次量, 相加即得界面反射振幅 $|r01|$ 、

相减即得光 2 的振幅 $|B|$ —— 这正是包络线法能把两个振幅拆开的原因。

"""

```

sg = np.linspace(1250.0, 2650.0, 4000)
n = 2.30 + 2.0e-5 * (sg - 1250.0)                # 缓慢的正常色散
K = np.sqrt(n ** 2 - (N0 * np.sin(np.radians(THETA_DEG))) ** 2)
r01 = (n - N0) / (n + N0)                         # 界面反射振幅
B = 0.11 * np.exp(-(sg - 1250.0) / 2600.0)        # 光 2 振幅, 随波数缓降
phi = 4.0 * np.pi * sg * (D_UM * 1e-4) * K
R = (r01 ** 2 + B ** 2 + 2 * r01 * B * np.cos(phi)) * 100
Rmax = (r01 + B) ** 2 * 100
Rmin = (r01 - B) ** 2 * 100

s0 = 1900.0                                        # 取一处作标注
i0 = int(np.argmin(np.abs(sg - s0)))

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 6.6), sharex=True)

# 上图: 反射率与包络线
ax1.plot(sg, R, lw=0.9, color='#1f4e79', label='两光束干涉谱 R(σ)', zorder=2)
ax1.plot(sg, Rmax, lw=1.6, ls='--', color='#d62728', label='上包络线 Rmax(σ)')

```

```

ax1.plot(sg, Rmin, lw=1.6, ls='—', color='#2ca02c', label='下包络线 Rmin( $\sigma$ )')
ax1.plot([sg[i0]] * 2, [Rmin[i0], Rmax[i0]], color='0.45', ls=':', lw=1.0)
ax1.plot(sg[i0], Rmax[i0], 'o', ms=7, color='#d62728', zorder=4,
         markeredgecolor='white', markeredgewidth=1.2)
ax1.plot(sg[i0], Rmin[i0], 'o', ms=7, color='#2ca02c', zorder=4,
         markeredgecolor='white', markeredgewidth=1.2)
ax1.text(0.985, 0.035, '竖虚线：在同一波数读取一对包络值',
         transform=ax1.transAxes, ha='right', va='bottom',
         fontsize=10, color='0.25', bbox=WHITE_BOX)
ax1.set_ylabel('反射率 R / %')
ax1.legend(fontsize=9, loc='upper right')
ax1.grid(alpha=0.3)

# 下图：开方后的上下包络与中线 u
sq_max, sq_min = np.sqrt(Rmax / 100), np.sqrt(Rmin / 100)
u = (sq_max + sq_min) / 2.0
ax2.plot(sg, sq_max, lw=1.6, ls='—', color='#d62728', label='√Rmax( $\sigma$ )')
ax2.plot(sg, sq_min, lw=1.6, ls='—', color='#2ca02c', label='√Rmin( $\sigma$ )')
ax2.plot(sg, u, lw=2.0, color='#1f4e79',
         label='u( $\sigma$ ) = (√Rmax+√Rmin)/2 = |r01|')
ax2.plot([sg[i0]] * 2, [sq_min[i0], sq_max[i0]], color='0.45', ls=':', lw=1.0)
ax2.plot(sg[i0], u[i0], 'o', ms=7, color='#1f4e79', zorder=4,
         markeredgecolor='white', markeredgewidth=1.2)
ax2.set_xlabel('波数  $\sigma$  / cm-1')
ax2.set_ylabel('反射振幅')
ax2.legend(fontsize=9, loc='center right')
ax2.grid(alpha=0.3)

fig.tight_layout()
name = '图9 包络线法（由干涉条纹的幅度提取界面反射信息）'
fig.savefig(OUT_DIR / f'{name}.svg', dpi=300)
fig.savefig(OUT_DIR / f'{name}.png', dpi=200)
plt.close(fig)
print(f'已保存 {OUT_DIR / name}.svg')
print(f' $\sigma$ ={sg[i0]:.0 f} cm-1 处: u={u[i0]:.4 f}, '
      f'|B|={(sq_max[i0]-sq_min[i0])/2:.4 f}')

if __name__ == '__main__':
    main() # 图2 (5.1.3)
    plot_polarization() # 图3 (5.1.2)
    plot_envelope_demo() # 图9 (5.1.2)

```

5.2 画图.py

```

# -*- coding: utf-8 -*-
"""

```

问题二结果可视化 —— 独立绘图脚本

调用 5.2求解.py 的算法函数，生成论文 5.2 节建议的插图：

- 图5 原始反射光谱（含异常首点，5.2.1）
- 图6 剔除异常点后的反射光谱及干涉峰谷标记（5.2.2）
- 图7 反推折射率 $n(\sigma)$ 随波数变化（含理论范围筛选，5.2.3）
- 图8 全谱模型拟合结果对比（含残差，5.2.4）
- 图9 双入射角厚度估计对比（散点 + 中位数 ± MAD，5.2.6）

用法:

```
python 5.2画图.py          # 输出到 ../ 图片/ (复现2/图片/)  
python 5.2画图.py DIR      # 输出到指定目录
```

运行环境: Python 3 + numpy / scipy / matplotlib

```
import sys  
import importlib.util  
from pathlib import Path  
import numpy as np  
import matplotlib  
matplotlib.use('Agg')  
import matplotlib.pyplot as plt  
  
def _load_solver():  
    """按文件名加载 5.2求解.py (文件名以数字开头, 不能用普通 import)。"""  
    p = Path(__file__).resolve().parent / '5.2求解.py'  
    spec = importlib.util.spec_from_file_location('q2_solver', p)  
    mod = importlib.util.module_from_spec(spec)  
    sys.modules['q2_solver'] = mod  
    spec.loader.exec_module(mod)  
    return mod
```

```
_q = _load_solver()  
load_data = _q.load_data  
remove_outliers = _q.remove_outliers  
detect_extrema = _q.detect_extrema  
solve = _q.solve  
fit_full_spectrum = _q.fit_full_spectrum  
two_beam_model_curve = _q.two_beam_model_curve
```

```
# 字体设置: 中文 SimSun (宋体), 西文/数字 Times New Roman, 数学 stix  
plt.rcParams['font.family'] = ['Times New Roman', 'SimSun']  
plt.rcParams['mathtext.fontset'] = 'stix'  
plt.rcParams['axes.unicode_minus'] = False
```

```
# 附件与入射角  
FILES = {'附件1 (入射角10°)': '题目/附件/附件1.xlsx',  
         '附件2 (入射角15°)': '题目/附件/附件2.xlsx'}  
THETA = {'附件1 (入射角10°)': 10., '附件2 (入射角15°)': 15.}  
COLORS = {'附件1 (入射角10°)': '#1f77b4', '附件2 (入射角15°)': '#d62728'}
```

```
BASE_DIR = Path(__file__).resolve().parent.parent.parent # 复现2 根目录
```

----- 图 5

```
def plot_raw_spectrum(raws, out_dir):  
    """图5 (5.2.1): 原始反射光谱 (全谱趋势), 黑色圆点标出异常首点位置。"""  
    fig, ax = plt.subplots(figsize=(8, 4.8))  
    for name, (s_raw, R_raw) in raws.items():  
        ax.plot(s_raw, R_raw, lw=0.7, color=COLORS[name], label=name, zorder=3)  
    # 标注首点零值 (R=0.0000%)  
    for name, (s_raw, R_raw) in raws.items():
```

```

i0 = np.where(R_raw <= 0)[0]
if len(i0):
    ax.plot(s_raw[i0[0]], 0, 'ko', ms=6, zorder=5)
ax.set_xlabel('波数  $\sigma$  /  $\text{cm}^{-1}$ ')
ax.set_ylabel('反射率 R / %')
ax.set_xlim(400, 4000)
ax.legend(fontsize=8, ncol=2, loc='upper right')
ax.grid(alpha=0.3)
fig.tight_layout()
_save(fig, out_dir, '图5 原始反射光谱')

```

----- 图 6

```

def plot_clean_spectrum(results, out_dir):
    """图6 (5.2.2)：聚焦有效区的反射光谱，峰谷标记仅标出
    通过理论范围筛选（规则□）后参与计算的峰谷对。"""
    fig, ax = plt.subplots(figsize=(8, 4.8))
    for name, (_s, _R, E, P) in results.items():
        m = (_s >= 1200) & (_s <= 2600)
        ax.plot(_s[m], _R[m], lw=0.7, color=COLORS[name], label=name, zorder=3)
        typ = {sig: t for sig, r, t in E}
        pm = {'max': ('^', 'tab:red'), 'min': ('v', 'tab:green')}
        for sp, sv in P[:, [0, 1]]:
            for s0 in (sp, sv):
                mk, c = pm[typ.get(s0, 'min')]
                ax.plot(s0, np.interp(s0, _s, _R), mk, ms=4.0,
                        color=c, zorder=4)
    ax.set_xlabel('波数  $\sigma$  /  $\text{cm}^{-1}$ ')
    ax.set_ylabel('反射率 R / %')
    ax.set_xlim(1200, 2600)
    ax.set_ylim(0, 25)
    ax.legend(fontsize=8, ncol=2, loc='lower right')
    ax.grid(alpha=0.3)
    fig.tight_layout()
    _save(fig, out_dir, '图6 剔除异常点后的反射光谱及干涉峰谷标记')

```

----- 图 7

```

def plot_refractive_index(results, out_dir):
    """图7 (5.2.3)：通过理论范围筛选后的反推折射率  $n(\sigma)$ ——散点为表 6
    对应数据，实线为线性插值曲线，绿色横带为理论折射率范围 2.20~2.60，
    虚线为文献给出的下限 2.20 与国标名义值 2.55。"""
    fig, ax = plt.subplots(figsize=(8, 4.8))
    ax.axhspan(2.20, 2.60, color='#cde7c0', alpha=0.55,
              label='理论折射率范围 2.20~2.60')
    for name, (_s, _R, _E, P) in results.items():
        o2 = np.argsort(P[:, 2])
        sc, nv = P[o2, 2], P[o2, 3]
        ax.scatter(sc, nv, s=26, color=COLORS[name], edgecolor='k', lw=0.4,
                  label=f'{name} (计算值)', zorder=3)
        sg = np.linspace(sc.min(), sc.max(), 200)
        ax.plot(sg, np.interp(sg, sc, nv), color=COLORS[name], lw=1.3,
                label=f'{name} (推测值)', zorder=4)
    ax.axhline(2.55, color='k', lw=1, ls='--', label='国家标准名义折射率 n=2.55')
    ax.set_xlabel('波数  $\sigma$  /  $\text{cm}^{-1}$ ')
    ax.set_ylabel('折射率 n')

```

```

ax.set_xlim(1200, 2600)
ax.set_ylim(2.1, 2.65)
ax.legend( fontsize=8)
ax.grid(alpha=0.3)
fig.tight_layout()
_save(fig, out_dir, '图7 反推折射率随波数的变化')

```

----- 图 8

```

def plot_full_fit(name, ang, s, R, theta, d_fit, rmse, out_dir):
    """图8 (5.2.4)：全谱模型拟合结果对比（附残差）。实测反射率取
    5.2.2 剔除异常点后的数据；d_fit 与 rms 放入图例标题。"""
    sg, Rm, Rg = two_beam_model_curve(s, R, theta, d_fit * 1e-4)
    fig, ax = plt.subplots(figsize=(8, 4.8))
    ax.plot(sg, Rg, lw=1.0, color='0.35', label='实测反射率')
    ax.plot(sg, Rm, lw=1.4, ls='--', color='#d62728', label='平均近似模型拟合（式(5)）')
    ax.set_xlabel('波数  $\sigma$  /  $\text{cm}^{-1}$ ')
    ax.set_ylabel('反射率 R / %')
    ax.set_xlim(sg.min(), sg.max())
    ax.legend(fontsize=9, title=f'拟合厚度 d={d_fit:.2f}  $\mu\text{m}$ , 残差 rms={rmse:.2f}%',
              title_fontsize=9)
    ax.grid(alpha=0.3)
    # 残差子图（右侧纵轴，浅色弱化，避免影响主图观感）
    ax2 = ax.twinx()
    ax2.plot(sg, Rg - Rm, lw=0.7, color='#c8c8c8')
    ax2.set_ylabel('残差 R_obs - R_fit / %', color='#a8a8a8')
    ax2.tick_params(axis='y', labelcolor='#a8a8a8')
    ax2.axhline(0, color='#dcdcdc', lw=0.6, ls=':')
    rmax = np.max(np.abs(Rg - Rm)) * 1.3
    ax2.set_ylim(-rmax, rmax)
    fig.tight_layout()
    _save(fig, out_dir, f'图8 全谱拟合结果（{name.split("(")[0]}, {ang:.0f}°）')

```

----- 图 9

```

def plot_two_angle(results, out_dir):
    """图9 (5.2.6)：双入射角厚度估计对比——散点为各峰谷对 d_j,
    实线为中位数，阴影带为中位数  $\pm$  MAD；两组带重合直观展示结果一致。"""
    from scipy.stats import median_abs_deviation
    from matplotlib.lines import Line2D
    from matplotlib.patches import Patch
    rng = np.random.default_rng(0) # 固定抖动种子，保证可复现
    fig, ax = plt.subplots(figsize=(7, 4.8))
    for i, (name, (_s, _R, _E, P)) in enumerate(results.items()):
        d = P[:, 6]
        med = np.median(d)
        mad = median_abs_deviation(d, scale=1.0)
        x = i + 1
        # 中位数  $\pm$  MAD 阴影带
        ax.fill_between([x - 0.28, x + 0.28], med - mad, med + mad,
                        color=COLORS[name], alpha=0.18)
        # 各峰谷对 d_j 散点（水平抖动避免重叠）
        jitter = rng.uniform(-0.09, 0.09, len(d))
        ax.scatter(x + jitter, d, s=32, color=COLORS[name],
                   edgecolor='k', lw=0.4, zorder=3)
        # 中位数横线与数值标注

```

```

ax.hlines (med, x - 0.28, x + 0.28, color=COLORS[name], lw=2.2,
           zorder=4)
ax.text (x, med + 0.3, f'{med:.2f}', ha='center',
         fontsize=9, color=COLORS[name])
ax.set_xticks ([1, 2])
ax.set_xticklabels (list (results.keys()))
ax.set_ylabel ('厚度估计 d_j / μm')
ax.set_xlim (0.5, 2.5)
ax.set_ylim (6, 14.5)
handles = [
    Line2D([0], [0], marker='o', color='w', markerfacecolor='0.5',
           markeredgcolor='k', markersize=7, label='各峰谷对厚度估计 d_j'),
    Line2D([0], [0], color='0.5', lw=2, label='中位数'),
    Patch (facecolor='0.5', alpha=0.25, label='中位数 ± MAD'),
]
ax.legend(handles=handles, fontsize=8, loc='upper right')
ax.grid(alpha=0.3, axis='y')
fig.tight_layout()
_save(fig, out_dir, '图9 双入射角厚度估计对比')

```

----- 工具

```

def _save(fig, out_dir, name):
    """同时保存 svg (论文嵌入) 与 png (快速预览) 。"""
    fig.savefig (out_dir / f'{name}.svg', dpi=300)
    fig.savefig (out_dir / f'{name}.png', dpi=200)
    plt.close (fig)
    print(f' 已保存 {out_dir / name}.svg')

def main():
    out_dir = Path(sys.argv[1]) if len(sys.argv) > 1 else BASE_DIR / '图片'
    out_dir.mkdir(parents=True, exist_ok=True)
    print(f'问题二结果可视化: 输出目录 {out_dir}')

    raws = {}          # 原始数据 (图5)
    results = {}        # 处理后数据 + 求解结果 (图6/7/9)
    for name, path in FILES.items():
        path = str(BASE_DIR / path)
        s_raw, R_raw = load_data(path)
        s, R = remove_outliers(s_raw, R_raw)
        th = np.radians (THETA[name])
        P, E = solve(s, R, th)          # 方案二: 筛选 + 插值
        med = np.median(P[:, 6])
        d_fit, rmse = fit_full_spectrum (s, R, th, med * 1e-4) # 方案一
        raws[name] = (s_raw, R_raw)
        results [name] = (s, R, E, P)
        plot_full_fit (name, THETA[name], s, R, th, d_fit, rmse, out_dir) # 图8

    plot_raw_spectrum(raws, out_dir)      # 图5 (5.2.1)
    plot_clean_spectrum (results, out_dir) # 图6 (5.2.2)
    plot_refractive_index (results, out_dir) # 图7 (5.2.3)
    plot_two_angle (results, out_dir)      # 图9 (5.2.6)
    print('全部插图生成完毕。')

```

```
if __name__ == '__main__':
    main()
```

5.2 可靠性核查.py

```
# -*- coding: utf-8 -*-
"""5.2.6 可靠性分析的核查脚本：
1) 稳健统计量（中位数 / MAD / 极值）
2) 共同波段两角折射率差  $\Delta n$  的均值与方向一致性
3) 全谱拟合残差：RMS、线性趋势斜率、滞后1自相关（结构性检验）
4) 联合拟合：两角共享厚度 d，与独立拟合比较（残差与 AIC）
"""

import importlib.util
import numpy as np
from pathlib import Path
from scipy.optimize import least_squares
from scipy.stats import median_abs_deviation

here = Path(__file__).resolve().parent
spec = importlib.util.spec_from_file_location('sol', here / '5.2求解.py')
sol = importlib.util.module_from_spec(spec)
spec.loader.exec_module(sol)

ROOT = here.parent.parent
FILES = {'附件1': ('题目/附件/附件1.xlsx', 10.),
         '附件2': ('题目/附件/附件2.xlsx', 15.)}

def prep(name, path, deg):
    s_raw, R_raw = sol.load_data(str(ROOT / path))
    s, R = sol.remove_outliers(s_raw, R_raw)
    th = np.radians(deg)
    P, E = sol.solve(s, R, th)
    return s, R, th, P

# ----- 1) 稳健统计量 -----
res = {}
print('=' * 70)
print('1) 方案二稳健统计量')
for name, (path, deg) in FILES.items():
    s, R, th, P = prep(name, path, deg)
    d_j = P[:, 6]
    med, mad = np.median(d_j), median_abs_deviation(d_j, scale=1.0)
    res[name] = dict(s=s, R=R, th=th, P=P, d_j=d_j, med=med, mad=mad)
    print(f' {name}({deg:.0f}°): 中位数 {med:.2f} μm, MAD {mad:.2f} μm '
          f'({mad/med*100:.1f}%), d_j 范围 {d_j.min():.2f}~{d_j.max():.2f} μm')
m1, m2 = res['附件1']['med'], res['附件2']['med']
print(f' 两中位数相对差异: {abs(m1-m2)/((m1+m2)/2)*100:.2f}%')

# ----- 2) 折射率一致性  $\Delta n$  -----
print('=' * 70)
print('2) 共同波段折射率差  $\Delta n = n(15^\circ) - n(10^\circ)$ ')
P1, P2 = res['附件1']['P'], res['附件2']['P']
c1, n1 = P1[:, 2], P1[:, 3]
c2, n2 = P2[:, 2], P2[:, 3]
lo = max(c1.min(), c2.min())
hi = min(c1.max(), c2.max())
```

```

grid = np.linspace(lo, hi, 200)
dn = np.interp(grid, c2, n2) - np.interp(grid, c1, n1)
base = np.interp(grid, c1, n1)
print(f' 共同波段: {lo:.1 f} ~ {hi:.1 f} cm-1')
print(f' 平均 Δn = {dn.mean():.4 f} ({dn.mean()/base.mean()*100:.2 f}%)')
print(f' Δn 符号一致(全为正)的比例: {(dn > 0).mean()*100:.0 f}%)')

# ----- 3) 残差结构检验 (独立拟合) -----
print('=' * 70)
print('3) 方案一独立拟合残差结构')
fits = {}
for name, (path, deg) in FILES.items():
    s, R, th, P = res[name]['s'], res[name]['R'], res[name]['th'], res[name]['P']
    med = res[name]['med']
    d_fit, rmse = sol.fit_full_spectrum(s, R, th, med * 1e-4)
    sg, Rm, Ro = sol.two_beam_model_curve(s, R, th, d_fit * 1e-4)
    resid = Ro - Rm
    # 线性趋势
    k = np.polyfit(sg, resid, 1)
    # 去趋势后滞后 1 自相关
    rt = resid - np.polyval(k, sg)
    ac = np.corrcorrcoef(rt[:-1], rt[1:])[0, 1]
    fits[name] = (d_fit, rmse)
    print(f' {name}: d_fit = {d_fit:.2 f} μm, RMS = {rmse:.3 f}%, '
          f'趋势斜率 = {k[0]:.3 e} %/cm-1, 滞后1自相关 = {ac:.3 f}%)

# ----- 4) 联合拟合: 共享厚度 -----
print('=' * 70)
print('4) 联合拟合 (两角共享 d) vs 独立拟合')
reg = {n: sol.fit_region_params(res[n]['s'], res[n]['R']) for n in FILES}

def joint_resid(d):
    out = []
    for (name, (path, deg)), (sg, Rg, nE, bE) in zip(FILES.items(), reg.values()):
        th = np.radians(deg)
        rs, rp = sol._fresnel_mod(nE, th)
        phi = 4 * np.pi * d * sg * np.sqrt(nE ** 2 - np.sin(th) ** 2)
        Rm = 0.5 * (rs ** 2 + bE ** 2 - 2 * rs * bE * np.cos(phi)) \
            + 0.5 * (rp ** 2 + bE ** 2 - 2 * rp * bE * np.cos(phi))
        out.append(Rm - Rg)
    return np.concatenate(out)

d0 = np.mean([fits[n][0] for n in FILES]) * 1e-4
solj = least_squares(joint_resid, d0, bounds=sol.D_BOUND)
d_joint = solj.x[0] * 1e4
r_all = joint_resid(solj.x[0])
N = len(r_all)
n1p = len(reg['附件1'][0]); n2p = len(reg['附件2'][0])
rms_j = np.sqrt(np.mean(r_all[n1p]**2)) * 100, np.sqrt(np.mean(r_all[n1p:]**2)) * 100
print(f' 联合拟合: d = {d_joint:.2 f} μm')
print(f' 联合拟合分角 RMS: 附件1 = {rms_j[0]:.3 f}%, 附件2 = {rms_j[1]:.3 f}%)')
print(f' 独立拟合分角 RMS: 附件1 = {fits["附件1"][1]:.3 f}%, 附件2 = {fits["附件2"][1]:.3 f}%)')
rss_j = np.sum(r_all**2)
rss_i = sum((fits[n][1]/100)**2 * len(reg[n][0]) for n in FILES)
aic = lambda rss, k: N * np.log(rss / N) + 2 * k
bic = lambda rss, k: N * np.log(rss / N) + k * np.log(N)

```

```
print(f' 独立拟合(2个d): AIC = {aic( rss_i , 3):.1 f}, BIC = {bic( rss_i , 3):.1 f}')
print(f' 联合拟合(1个d): AIC = {aic( rss_j , 2):.1 f}, BIC = {bic( rss_j , 2):.1 f}')
print(f' 联合拟合 d 与两角独立值({fits["附件1"][0]:.2 f}, { fits ["附件2"][0]:.2 f}) 的偏差: '
      f' {abs( d_joint-fits ["附件1"][0])/ d_joint*100:.2f}%, '
      f' {abs( d_joint-fits ["附件2"][0])/ d_joint*100:.2f}%')
```

5.2 求解.py

```
# -*- coding: utf-8 -*-
"""
```

碳化硅外延层厚度确定 —— 问题二求解算法（与论文 5.2 节一致）

数据预处理（5.2.2）执行两条剔除规则：

- 删除反射率 $R \leq 0\%$ 的数据点
反射率不可能取零或负值，故 $R \leq 0$ 的点不可能是任何波段上的真实读数。
本批数据中这样的点每份附件恰有一个（ $\sigma=399.68 \text{ cm}^{-1}$ ， $R=0.0000\%$ ）。
- 删除反推折射率超出理论范围 2.20~2.60 的峰谷对
包络线法反推的 n 必须落在碳化硅红外透明区的理论折射率范围内
（下界 2.20、上界 2.60，由文献给出，见论文 5.2.2）；
超出该范围的反推 n 在物理上不可能，是残余射线带及其邻域两光束模型前提失效的产物，对应的峰谷对不参与后续求解。

方案二（5.2.3，相邻峰谷法）在通过上述筛选的峰谷对上进行：

1. 逐对相邻峰谷用包络线法反推折射率（问题一式(13)(19)）
2. 剔除 n 超出理论范围的峰谷对
3. 由各对反推值按中心波数线性插值构造 $n(\sigma)$ ，在峰、谷波数处分别取 $n(\sigma_p)$ 、 $n(\sigma_v)$
4. 代入厚度公式（问题一式(5)）得 d_j ，取中位数与 MAD

方案一（5.2.4，全谱拟合）：以方案二所得 d 为初值，对全谱作两光束模型最小二乘拟合， $n(\sigma)$ 、 $B(\sigma)$ 由包络线法散点线性插值给出，作相互验证。

运行环境：Python 3 + numpy / scipy / openpyxl

用法：在项目根目录运行 `python "5 模型的建立与求解/5.2 问题2/5.2求解.py"`

```
"""
```

```
import numpy as np
from scipy.signal import find_peaks
from scipy.stats import median_abs_deviation
import openpyxl
```

```
# 常数
```

```
N0 = 1.0          # 空气折射率 n0
PROM = 1.0        # 峰谷检测的显著度门槛（百分点）
FULL_RANGE = (400., 4000.) # 参与峰谷检测的波数范围（剔除异常点后的全谱）
N_RANGE = (2.20, 2.60) # 折射率的理论范围（文献给出的 SiC 红外折射率，见 5.2.2）
FIT_RANGE = (1400.0, 2500.0) # 方案一拟合的波数范围（5.2.2 筛选后的有效光谱区）
RESTSTRAHLEN = (787.0, 968.0) # 碳化硅残余射线带（仅绘图/讨论用）
D_BOUND = (4e-4, 3e-3) # 厚度拟合边界（cm），即 4~30 μm
```

```
# ----- 数据读取与预处理
```

```
def load_data(path):
    """读取xlsx 附件，返回(波数  $\text{cm}^{-1}$ , 反射率 %)。"""
    wb = openpyxl.load_workbook(path, read_only=True)
```

```

rows = list (wb.active.iter_rows (values_only=True))
wb.close()
arr = np.array ([[ float (r[0]), float (r[1])] for r in rows[1:]
                  if r[0] is not None and r[1] is not None])
return arr[:, 0], arr[:, 1]

def remove_outliers(s, R):
    """剔除规则：删除反射率  $R \leq 0\%$  的数据点（论文 5.2.2）。"""
    keep = R > 0
    return s[keep], R[keep]

# ----- 峰谷检测
def detect_extrema(s, R, prom=PROM):
    """检测干涉极值；prom 为显著度（%），用于剔除噪声峰。"""
    pk, _ = find_peaks(R, prominence=prom)
    va, _ = find_peaks(-R, prominence=prom)
    E = sorted([(s[i], R[i], 'max') for i in pk]
               + [(s[i], R[i], 'min') for i in va])
    return E

# ----- 包络线法与厚度
def envelope_pairs(E, lo=FULL_RANGE[0], hi=FULL_RANGE[1]):
    """逐对相邻峰谷用包络线法反推折射率。

    依据问题一式(13)(19):
         $u = (\sqrt{R_{\max}} + \sqrt{R_{\min}}) / 2$ ,  $n = n_0 * (1+u) / (1-u)$ 
    返回数组，列为 [ $\sigma_p$ ,  $\sigma_v$ ,  $\sigma$ 中心,  $n(\sigma$ 中心)].
    """
    rows = []
    for a, b in zip(E[:-1], E[1:]):
        if a[2] == b[2]: # 必须峰谷相邻
            continue
        s_p, s_v = min(a[0], b[0]), max(a[0], b[0])
        if not (lo < s_p and s_v < hi): # 该对须完整落在范围内
            continue
        Rmax, Rmin = max(a[1], b[1]), min(a[1], b[1])
        u = (np.sqrt(Rmax / 100.) + np.sqrt(Rmin / 100.)) / 2. # 式(13)
        if not (0 < u < 1):
            continue
        n = N0 * (1 + u) / (1 - u) # 式(19)
        rows.append((s_p, s_v, (s_p + s_v) / 2., n))
    return np.array(rows)

def envelope_method(E, lo=FULL_RANGE[0], hi=FULL_RANGE[1]):
    """包络线法（含 |B| 反推），列为 [ $\sigma$ 中心, n, r01, B]。"""
    out = []
    for a, b in zip(E[:-1], E[1:]):
        if a[2] == b[2]:
            continue
        sl, sh = min(a[0], b[0]), max(a[0], b[0])
        if not (lo < sl and sh < hi):
            continue

```



```

Rmax, Rmin = max(a[1], b[1]), min(a[1], b[1])
if Rmin < 0.5 or Rmax > 90: # 剔除近零谷与饱和峰
    continue
sr, srm = np.sqrt(Rmax / 100.), np.sqrt(Rmin / 100.)
r01 = (sr + srm) / 2. # 式(13)
B = (sr - srm) / 2. # 式(12)
if not (0 < r01 < 1):
    continue
n = N0 * (1 + r01) / (1 - r01) # 式(19)
out.append(((sl + sh) / 2., n, r01, B))
return np.array(out)

def thickness_pair(s_p, s_v, n_p, n_v, theta):
    """问题一式(5): 严格计入折射率色散, 分别取 n(σp)、n(σv)。

    d = 1 / (4*σv*sqrt(n(σv)^2-sin^2θ) - σp*sqrt(n(σp)^2-sin^2θ))
    返回厚度, 单位 cm。
    """
    c_p = np.sqrt(n_p ** 2 - np.sin(theta) ** 2)
    c_v = np.sqrt(n_v ** 2 - np.sin(theta) ** 2)
    return 1.0 / (4.0 * abs(s_v * c_v - s_p * c_p))

def filter_pairs(P, nrange=N_RANGE):
    """剔除反推折射率超出理论范围的峰谷对(论文 5.2.2 第二条规则)。

    P 列为 [σp, σv, σ中心, n反推]。反推 n 必须落在碳化硅红外透明区的
    理论折射率范围 [2.20, 2.60] 内, 超出即判为包络线法前提失效。
    """
    n = P[:, 3]
    keep = (n >= nrange[0]) & (n <= nrange[1])
    return P[keep], np.count_nonzero(~keep)

def solve(s, R, theta, lo=FULL_RANGE[0], hi=FULL_RANGE[1]):
    """方案二完整求解: 峰谷检测 → 包络线法 → 理论范围筛选 → 逐对厚度。

    n(σ) 由各对反推值按中心波数线性插值得到, 再在 σp、σv 处分别取值。
    返回数组, 列为 [σp, σv, σ中心, n反推, n(σp), n(σv), d_j/μm]。
    """
    E = detect_extrema(s, R)
    P = envelope_pairs(E, lo, hi)
    P, _n_drop = filter_pairs(P)
    o = np.argsort(P[:, 2])
    n_p = np.interp(P[:, 0], P[o, 2], P[o, 3]) # n(σp)
    n_v = np.interp(P[:, 1], P[o, 2], P[o, 3]) # n(σv)
    d = thickness_pair(P[:, 0], P[:, 1], n_p, n_v, theta) * 1e4 # cm → μm
    return np.column_stack([P, n_p, n_v, d]), E

# ----- 方案一: 有效区全谱拟合
def _fresnel_mod(nE, theta):
    """双偏振菲涅耳反射系数模值(问题一式(15)(16)), 返回 (|rs|, |rp|)。

    干涉项统一取负 (-2|r| • B • cosφ): 空气—外延层界面的反射系数与衬底

```

反射的符号共同给出第二束光相对第一束光的 π 相位，使反射率极大出现在 $\cos\varphi=-1$ 处（见 5.1 式(5) 的符号讨论）。

"""

```
sin2 = np.sin(theta) ** 2
cos1 = np.sqrt(nE ** 2 - sin2) / nE
c0 = np.cos(theta)
n0c, nc1 = N0 * c0, nE * cos1
nc0, n0c1 = nE * c0, N0 * cos1
rs = np.abs((n0c - nc1) / (n0c + nc1))
rp = np.abs((nc0 - n0c1) / (nc0 + n0c1))
return rs, rp
```

```
def envelope_pairs_full(E, lo=FIT_RANGE[0], hi=FIT_RANGE[1]):
    """逐对反推并通过理论范围筛选，返回 [σ, σv, σ中心, n, B]。"""
    rows = []
    for a, b in zip(E[:-1], E[1:]):
        if a[2] == b[2]:
            continue
        sp, sv = min(a[0], b[0]), max(a[0], b[0])
        if not (lo < sp and sv < hi):
            continue
        Rmax, Rmin = max(a[1], b[1]), min(a[1], b[1])
        sr, srm = np.sqrt(Rmax / 100.), np.sqrt(Rmin / 100.)
        u = (sr + srm) / 2. # 式(13)
        if not (0 < u < 1):
            continue
        n = N0 * (1 + u) / (1 - u) # 式(19)
        if not (N_RANGE[0] <= n <= N_RANGE[1]): # 规则□ 筛选
            continue
        B = (sr - srm) / 2. # 式(12)
        rows.append((sp, sv, (sp + sv) / 2., n, B))
    return np.array(rows)
```

```
def fit_region_params(s, R, lo=FIT_RANGE[0], hi=FIT_RANGE[1]):
    """有效区拟合参数: (σ, R_obs/100, n(σ), B(σ)), 由筛选后散点线性插值。"""
    E = detect_extrema(s, R)
    pt = envelope_pairs_full(E, lo, hi)
    pt = pt[np.argsort(pt[:, 2])]
    m = (s >= lo) & (s <= hi)
    sg, Rg = s[m], R[m] / 100.
    nE = np.interp(sg, pt[:, 2], pt[:, 3])
    bE = np.interp(sg, pt[:, 2], pt[:, 4])
    return sg, Rg, nE, bE
```

```
def two_beam_model_curve(s, R, theta, d, polarized=True):
    """给定厚度 d(cm)，返回有效区 (σ, R_model/%, R_obs/%)。"""
    sg, Rg, nE, bE = fit_region_params(s, R)
    sin2 = np.sin(theta) ** 2
    phi = 4 * np.pi * d * sg * np.sqrt(nE ** 2 - sin2)
    if polarized:
        rs, rp = _fresnel_mod(nE, theta)
        Rm = 0.5 * (rs ** 2 + bE ** 2 - 2 * rs * bE * np.cos(phi)) \
            + 0.5 * (rp ** 2 + bE ** 2 - 2 * rp * bE * np.cos(phi))
```

```

else:
    r01 = (nE - N0) / (nE + N0)
    Rm = r01 ** 2 + bE ** 2 - 2 * r01 * bE * np.cos(phi)
    return sg, Rm * 100., Rg * 100.

def fit_full_spectrum(s, R, theta, d0, polarized=True):
    """(方案一)有效区最小二乘拟合, 返回(d_fit/μm, rms/%)."""
    sg, Rg, nE, bE = fit_region_params(s, R)
    sin2 = np.sin(theta) ** 2
    if polarized:
        rs, rp = _fresnel_mod(nE, theta)

        def model(d):
            phi = 4 * np.pi * d * sg * np.sqrt(nE ** 2 - sin2)
            return 0.5 * (rs ** 2 + bE ** 2 - 2 * rs * bE * np.cos(phi)) \
                + 0.5 * (rp ** 2 + bE ** 2 - 2 * rp * bE * np.cos(phi))
    else:
        r01 = (nE - N0) / (nE + N0)

        def model(d):
            phi = 4 * np.pi * d * sg * np.sqrt(nE ** 2 - sin2)
            return r01 ** 2 + bE ** 2 - 2 * r01 * bE * np.cos(phi)

    from scipy.optimize import least_squares
    resid = lambda d: model(d) - Rg
    sol = least_squares(resid, d0, bounds=D_BOUND)
    rmse = np.sqrt(np.mean(resid(sol.x[0]) ** 2)) * 100
    return sol.x[0] * 1e4, rmse

# ----- 主程序
def main():
    files = {'附件1 (入射角10°)': '题目/附件/附件1.xlsx',
            '附件2 (入射角15°)': '题目/附件/附件2.xlsx'}
    theta_deg = {'附件1 (入射角10°)': 10., '附件2 (入射角15°)': 15.}

    print('=' * 74)
    print('问题二计算结果 (数据预处理: □删除 R≤0% 的点; □剔除 n 超出理论范围的峰谷对)')
    print('=' * 74)

    summary = {}
    for name, path in files.items():
        s_raw, R_raw = load_data(path)
        s, R = remove_outliers(s_raw, R_raw)
        th = np.radians(theta_deg[name])

        P, E = solve(s, R, th) # 方案二: 相邻峰谷法
        # 重新统计筛选剔除的对数
        Pp = envelope_pairs(E)
        _, n_drop = filter_pairs(Pp)
        d_j = P[:, 6]
        med = np.median(d_j)
        mad = median_abs_deviation(d_j, scale=1.0)
        d_fit, rmse = fit_full_spectrum(s, R, th, med * 1e-4) # 方案一: 全谱拟合
        summary[name] = (med, mad, d_fit, rmse)

```

```

print(f'\n 【{name}】 ')
print(f' 数据点 {len(R_raw)} → {len(R)}')
f' (剔除  $\sigma=\{s\_raw[R\_raw \leq 0][0]:.2f\} \text{ cm}^{-1}$ ,  $R=\{R\_raw[R\_raw \leq 0][0]:.4f\}\%$ ) ')
print(f' 全谱检出干涉极值 {len(E)} 个'
f' (峰 {sum(1 for e in E if e[2] == "max")}, '
f' 谷 {sum(1 for e in E if e[2] == "min")}) , '
f' 峰谷对 {len(Pp)} 对 → 筛选 ( $n \in [2.20, 2.60]$ ) 剔除 {n_drop} 对, '
f' 保留 {len(P)} 对')
print(f'\n  {"sp":>9} {"sv":>9} {"n(sp)":>7} {"n(sv)":>7} {"d_j/μm":>9}')
print(f'  {"-" * 9} {"-" * 9} {"-" * 7} {"-" * 7} {"-" * 9}')
for sp, sv, mid, n, np_, nv, d in P:
    print(f'  {sp:.9.2f} {sv:.9.2f} {np_:.7.3f} {nv:.7.3f} {d:.9.2f}')
print(f'\n >> 方案二 (相邻峰谷法, 中位数) d = {med:.2f} μm')
print(f' >> 分散程度 MAD = {mad:.2f} μm ( {mad / med * 100:.1f}%) ')
print(f' >> d_j 范围 {d_j.min():.2f} ~ {d_j.max():.2f} μm')
print(f' >> 方案一 (全谱拟合) d = {d_fit:.2f} μm, rms = {rmse:.2f}%)

print('\n' + '=' * 74)
(d1, m1, f1, _), (d2, m2, f2, _) = list(summary.values())
rel = abs(d1 - d2) / ((d1 + d2) / 2) * 100
print(f' 双入射角一致性 (方案二): 10° 测得 {d1:.2f} μm, 15° 测得 {d2:.2f} μm, '
f' 相对差异 {rel:.2f} %')
print(f' 方案一拟合: {f1:.2f} μm / {f2:.2f} μm')
print(f' 最终结论: 外延层厚度  $d \approx \{np.mean([d1, d2]):.1f\} \mu m$ '
f' (单次测量的分散程度约  $\pm \{np.mean([m1, m2]):.1f\} \mu m$ ) ')
print('=' * 74)

if __name__ == '__main__':
    main()

```

5.3 Drude 求解.py

```

# -*- coding: utf-8 -*-
"""

```

问题三: 引入衬底 Drude 色散模型的硅外延层厚度求解

评议指出两处缺陷: (1) 方案一以常数折射率拟合, 低波数端失效后被直接放弃; (2) 方案二的相邻峰谷公式未计入衬底界面反射相移的色散 $d\delta_{12}/d\sigma$, 造成峰谷法与 FFT 之间约 1.2% 的系统偏差。本脚本一并修复。

物理模型

外延层折射率取文献名义值 $n = 3.42$ (弱吸收, $A \approx 1$)。重掺杂衬底的介电函数用 Drude 模型描述 (波数 σ 以 cm^{-1} 计):

$$\epsilon_2(\sigma) = \epsilon_\infty - \sigma_p^2 / (\sigma^2 + i\gamma\sigma),$$

σ_p 为等离子体波数, γ 为阻尼波数。衬底与外延层同为硅, 其高频介电常数与外延层一致, 取 $\epsilon_\infty \leq n^2 = 11.7$; 若放任 $\epsilon_\infty > n^2$, r_{12} 会在谱段内过零, δ_{12} 在过零点陡变, 修正后的峰值值在高波数端发散, 故将 ϵ_∞ 约束在该物理上限之内。代入三层膜 Airy 公式 (即论文式(30)), s 偏振反射率为:

$$r = (r_{01} + r_{12} e^{i\varphi}) / (1 + r_{01} r_{12} e^{i\varphi}),$$

$$\varphi = 4\pi\sigma d \sqrt{(n^2 - \sin^2\theta)}, \quad r_{12} = (\beta_1 - \beta_2)/(\beta_1 + \beta_2),$$

$$\beta_1 = \sqrt{(n^2 - \sin^2\theta)}, \quad \beta_2 = \sqrt{(\epsilon_2 - \sin^2\theta)}.$$

方案一（Drude 全谱拟合）：厚度初值不取峰间距估计（低波数端条纹被 δ_{12} 色散拉宽，该估计有偏），而用 FFT 基频独立锚定；拟合参数为 d 、 ϵ_{∞} 、 σ_p 、 γ ，拟合区间取全谱（400~4000 cm^{-1} ），不再截断低波数端。

方案二（相移修正的相邻峰谷法）：条纹极值条件为

$$\varphi(\sigma) + \delta_{12}(\sigma) = k\pi, \quad \delta_{12} = \arg r_{12},$$

相邻峰、谷之间相位差为 π ，逐对解出

$$d_j = [\pi - \delta_{12}(\sigma_2) + \delta_{12}(\sigma_1)] / [4\pi c (\sigma_2 - \sigma_1)],$$

$$c = \sqrt{(n^2 - \sin^2\theta)},$$

$\delta_{12}(\sigma)$ 由方案一拟合的 Drude 参数算出。 δ_{12} 为常数时即退化为原式。

相干长度定量校验：采样间隔 $\Delta\sigma$ 由数据估算，相干长度按 $L_c \approx 1/\Delta\sigma$ 计算，并与相邻光束光程差 $\Delta L = 2dc$ 对比。

运行环境：Python 3 + numpy / scipy / openpyxl

用法：在项目根目录运行 `python "5 模型的建立与求解/5.3 问题3/5.3Drude求解.py"`

```
import numpy as np
from scipy.optimize import least_squares
from scipy.signal import find_peaks
from scipy.stats import median_abs_deviation
import openpyxl

N0 = 1.0
N_SI = 3.42          # 外延层折射率文献名义值（红外透明区）
EI_INF = N_SI ** 2   # 衬底高频介电常数上限（同为硅，≈11.7）
PROM = 1.0           # 峰谷检测显著度门槛（百分点），与 5.2 节相同
D_BOUND = (5e-5, 3e-3) # 厚度拟合边界（cm），即 0.5~30 μm
S_CUT = 1500.        # 峰谷统计的低波数截断（cm-1）
```

```
def load_data(path):
    """读取 xlsx 附件，返回 (波数 cm-1, 反射率 %). """
    wb = openpyxl.load_workbook(path, read_only=True)
    rows = list(wb.active.iter_rows(values_only=True))
    wb.close()
    arr = np.array([[float(r[0]), float(r[1])] for r in rows[1:]
                    if r[0] is not None and r[1] is not None])
    return arr[:, 0], arr[:, 1]
```

```
def eps_sub(s, eps_inf, sigma_p, gamma):
    """衬底介电函数的 Drude 模型，σ 以 cm-1 计。"""
    return eps_inf - sigma_p ** 2 / (s ** 2 + 1j * gamma * s)
```

```
def r_airy(s, d, eps_inf, sigma_p, gamma, theta):
    """三层膜 Airy 反射振幅与衬底界面反射系数 (r, r12). """
```

```

sin2 = np.sin(theta) ** 2
beta1 = np.sqrt(N_SI ** 2 - sin2)
beta2 = np.sqrt(eps_sub(s, eps_inf, sigma_p, gamma) - sin2 + 0j)
r01 = (np.cos(theta) - beta1) / (np.cos(theta) + beta1)
r12 = (beta1 - beta2) / (beta1 + beta2)
e = np.exp(4j * np.pi * s * d * beta1)
return (r01 + r12 * e) / (1 + r01 * r12 * e), r12

def R_model(s, d, eps_inf, sigma_p, gamma, theta):
    """Drude 多光束反射率 (%)。"""
    r, _ = r_airy(s, d, eps_inf, sigma_p, gamma, theta)
    return np.abs(r) ** 2 * 100.

def d0_from_fft(s, R, theta):
    """FFT 基频锚定厚度初值 (cm)：截断、均匀化、去趋势、加窗、零填充。"""
    m = (R > 0) & (s >= S_CUT)
    sg = np.linspace(s[m].min(), s[m].max(), m.sum())
    Rg = np.interp(sg, s[m], R[m])
    Rg = Rg - np.polyval(np.polyfit(sg, Rg, 3), sg)
    n = len(Rg)
    sp = np.fft.rfft(Rg * np.hanning(n), n=8 * n)
    f0 = (np.argmax(np.abs(sp[1:])) + 1) / ((sg[-1] - sg[0]) * 8)
    c = np.sqrt(N_SI ** 2 - np.sin(theta) ** 2)
    return f0 / (2 * c)

def fit_drude(s, R, theta):
    """方案一：全谱 Drude 拟合，返回 (参数向量, 残差均方根/百分点)。
    参数顺序：d/cm,  $\epsilon_\infty$ ,  $\sigma_p$ ,  $\gamma$ 。
    分两步：先以 FFT 厚度为锚，在 ( $\sigma_p$ ,  $\gamma$ ,  $\epsilon_\infty$ ) 网格上多初值搜索，
    确定衬底色散背景；再全参数精修。"""
    d0 = d0_from_fft(s, R, theta)
    resid = lambda x: R_model(s, *x, theta) - R
    best = None
    for sp in (800., 1500., 2500., 3500., 4000.):
        for ga in (150., 400., 1000.):
            sol = least_squares(lambda y: R_model(s, d0, y[0], y[1], y[2],
                                                    theta) - R,
                                [EI_INF - 0.05, sp, ga],
                                bounds=(5., 200., 20.),
                                [EI_INF, 4000., 3000.]),
                                x_scale='jac')
            r = np.sqrt(np.mean((R_model(s, d0, *sol.x, theta) - R) ** 2))
            if best is None or r < best[0]:
                best = (r, d0, sol.x)
    x0 = [best[1], *best[2]]
    lo = [D_BOUND[0], 5., 200., 20.]
    hi = [D_BOUND[1], EI_INF, 4000., 3000.]
    sol = least_squares(resid, x0, bounds=(lo, hi), x_scale='jac')
    rms = np.sqrt(np.mean(resid(sol.x) ** 2))
    return sol.x, rms

def delta12(s, x, theta):

```

```

"""衬底界面反射相移  $\delta_{12}(\sigma)$ , 按波数展开。"""
sg = np.linspace(s.min(), s.max(), 4001)
_, r12 = r_airy(sg, x[0], x[1], x[2], x[3], theta)
return np.interp(s, sg, np.unwrap(np.angle(r12)))

def pv_pairs(path, theta, x):
    """一份附件的相邻峰谷对: 返回数组
    [ $\sigma_1$ ,  $\sigma_2$ ,  $d_j$ 未修正/ $\mu\text{m}$ ,  $d_j$ 修正/ $\mu\text{m}$ ]。"""
    s, R = load_data(path)
    m = R > 0
    s, R = s[m], R[m]
    pk, _ = find_peaks(R, prominence=PROM)
    va, _ = find_peaks(-R, prominence=PROM)
    E = sorted([(s[i], 'max') for i in pk] + [(s[i], 'min') for i in va])
    c = np.sqrt(N_SI ** 2 - np.sin(theta) ** 2)
    dl = delta12(s, x, theta)
    rows = []
    for a, b in zip(E[:-1], E[1:]):
        if a[1] == b[1]:
            continue
        s1, s2 = min(a[0], b[0]), max(a[0], b[0])
        if s1 < S_CUT:
            continue
        i1, i2 = np.argmin(np.abs(s - s1)), np.argmin(np.abs(s - s2))
        d_unc = 1e4 / (4. * c * (s2 - s1))
        d_cor = 1e4 * (np.pi - dl[i2] + dl[i1]) / (4. * np.pi * c * (s2 - s1))
        rows.append((s1, s2, d_unc, d_cor))
    return np.array(rows)

def coherence(s, d_um, theta):
    """相干长度定量校验: 返回(采样间隔,  $L_c/\mu\text{m}$ , 光程差/ $\mu\text{m}$ )。"""
    ds = np.median(np.diff(s))
    Lc = 1.0 / ds * 1e4 # cm  $\rightarrow \mu\text{m}$ 
    c = np.sqrt(N_SI ** 2 - np.sin(theta) ** 2)
    return ds, Lc, 2. * d_um * c

def carrier_density(sigma_p):
    """由等离子体波数估算衬底掺杂浓度 ( $\text{cm}^{-3}$ )。
     $N = (2\pi c \sigma_p)^2 \epsilon_0 m^* / e^2$ , 电导有效质量取  $m^* = 0.26 m_e$ 。"""
    eps0, e, me = 8.854e-12, 1.602e-19, 9.109e-31
    wp = 2. * np.pi * 2.998e10 * sigma_p #  $\text{cm}^{-1} \rightarrow \text{rad/s}$ 
    return wp ** 2 * eps0 * 0.26 * me / e ** 2 / 1e6

def main():
    files = [('附件 3 (Si, 10°)', '题目/附件/附件3.xlsx', np.radians(10.)),
             ('附件 4 (Si, 15°)', '题目/附件/附件4.xlsx', np.radians(15.))]

    print('=' * 76)
    print('方案一 (Drude 色散模型全谱拟合, 400~4000  $\text{cm}^{-1}$ )')
    print('=' * 76)
    fits = {}
    for name, path, theta in files:

```

```

s, R = load_data(path)
m = R > 0
x, rms = fit_drude(s[m], R[m], theta)
fits[name] = (s, x, theta)
print(f'{name}')
print(f' d = {x[0] * 1e4:.3 f} μm ε∞ = {x[1]:.2 f} ',
      f'σp = {x[2]:.0 f} cm-1 γ = {x[3]:.0 f} cm-1 rms = {rms:.2 f}%')
print(f' 屏蔽等离子体边缘 σp/√ε∞ = {x[2] / np.sqrt(x[1]):.0 f} cm-1',
      f' 衬底掺杂浓度 N ≈ {carrier_density(x[2]):.2 e} cm-3')

print('\n' + '=' * 76)
print('方案二（相邻峰谷法，含相移色散修正；δ12取自Drude拟合）')
print('=' * 76)
for name, path, theta in files:
    _s, x, _t = fits[name]
    P = pv_pairs(path, theta, x)
    med_u, mad_u = np.median(P[:, 2]), median_abs_deviation(P[:, 2])
    med_c, mad_c = np.median(P[:, 3]), median_abs_deviation(P[:, 3])
    print(f'\n【{name}】峰谷对 {len(P)} 对')
    print(f' {"σ1":>9} {"σ2":>9} {"d未修正":>8} {"d修正":>8}')
    for s1, s2, du, dc in P:
        print(f' {s1:.9.2 f} {s2:.9.2 f} {du:.8.3 f} {dc:.8.3 f}')
    print(f' 未修正：中位数 {med_u:.3 f} μm, MAD {mad_u:.3 f} μm'
          f' ( {mad_u / med_u * 100:.1 f}% ) ')
    print(f' 修正后：中位数 {med_c:.3 f} μm, MAD {mad_c:.3 f} μm'
          f' ( {mad_c / med_c * 100:.1 f}% ) ')

print('\n' + '=' * 76)
print('相干长度定量校验 (L_c ≈ 1/Δσ, Δσ为采样间隔)')
print('=' * 76)
for name, path, theta in files:
    s, x, _t = fits[name]
    ds, Lc, dL = coherence(s, x[0] * 1e4, theta)
    print(f'{name}<16} Δσ = {ds:.3 f} cm-1 L_c ≈ {Lc:.0 f} μm'
          f' ΔL = 2dc ≈ {dL:.1 f} μm ΔL/L_c ≈ {dL / Lc:.1 e}')
print('=' * 76)

if __name__ == '__main__':
    main()

```

5.3 厚度求解.py

```

# -*- coding: utf-8 -*-
"""

```

问题三：硅外延层厚度的多光束模型计算（方案一，与论文 5.3.5 节一致）

附件 3、4 已确认存在多光束干涉。方案一以式(30)的多光束反射率表达式对全谱作最小二乘拟合，显式计入后续各束反射光的贡献：

$$R(\sigma) = |r_{01} - B(\sigma) \cdot e^{i\varphi} / (1 - q \cdot e^{i\varphi})|^2,$$

$$\varphi = 4\pi\sigma \cdot d \cdot \sqrt{n^2 - n_0^2 \cdot \sin^2\theta}.$$

硅谱的第二束光振幅随波数变化明显（低波数端谷接近零，高波数端条纹变浅），故 B 不作常数，而取若干节点的线性插值；r01 与 q 仍为常数。

波数低于约 1000 cm^{-1} 的低频段，重掺杂衬底的折射率偏离名义值较多，常数折射率的前提不再成立，拟合取 $1000\sim 3400\text{ cm}^{-1}$ 的有效区间。

数据预处理与 5.2 节相同：删除反射率非正的点。

计算流程：

1. 折射率取硅的文献名义值 $n = 3.42$ （红外透明区）；
2. 厚度初值由相邻峰间距估算，并在其附近扫描残差精化；
3. 以式(30)作最小二乘拟合，拟合参数为厚度 d 、 B 的节点值与组合量 $q = r_{10} \cdot r_{12} \cdot A$ ；
4. 输出拟合厚度、 $|q|$ 与残差均方根。

运行环境：Python 3 + numpy / scipy / openpyxl

用法：在项目根目录运行 `python "5 模型的建立与求解/5.3 问题3/5.3厚度求解.py"`

"""

```
import numpy as np
from scipy.optimize import least_squares
from scipy.signal import find_peaks
import openpyxl

N0 = 1.0
N_SI = 3.42          # 硅的文献名义折射率（红外透明区）
D_BOUND = (5e-5, 3e-3) # 厚度拟合边界（cm），即 0.5~30 μm
KNOTS = 6           # B(σ) 插值节点数
FIT_RANGE = (1000., 3400.) # 有效拟合区间（cm-1）
PROM = 1.0          # 峰谷检测显著度门槛（百分点），与 5.2 节相同
```

```
def load_data(path):
    """读取 xlsx 附件，返回 (波数 cm-1, 反射率 %)。”"""
    wb = openpyxl.load_workbook(path, read_only=True)
    rows = list(wb.active.iter_rows(values_only=True))
    wb.close()
    arr = np.array([[float(r[0]), float(r[1])] for r in rows[1:]
                    if r[0] is not None and r[1] is not None])
    return arr[:, 0], arr[:, 1]
```

```
def phase(s, d, theta):
    """往返相位  $\varphi = 4\pi\sigma d \cdot \sqrt{n^2 - \sin^2\theta}$ 。”"""
    return 4 * np.pi * s * d * np.sqrt(N_SI**2 - np.sin(theta)**2)
```

```
def r_multi(s, d, knots, b_nodes, q, r01, theta):
    """多光束模型，式(30):  $R = |r_{01} - B(\sigma) e^{i\varphi} / (1 - q e^{i\varphi})|^2$ ,
    B(σ) 由节点值线性插值。”"""
    phi = phase(s, d, theta)
    b = np.interp(s, knots, b_nodes)
    return np.abs(r01 - b * np.exp(1j * phi)
                  / (1.0 - q * np.exp(1j * phi)))**2 * 100.
```

```
def fit(s, R, theta, model, x0, bounds=(-np.inf, np.inf)):
    """最小二乘拟合，返回参数向量与残差均方根（百分点）。”"""
    resid = lambda x: model(s, *x, theta) - R
    sol = least_squares(resid, x0, bounds=bounds, x_scale='jac')
```

```

rmse = np.sqrt(np.mean(resid(sol.x) ** 2))
return sol.x, rmse

def d0_from_peaks(s, R, theta):
    """相邻峰间距估厚度初值:  $\Delta\sigma \approx 1/(2d \cdot \sqrt{n^2 - \sin^2\theta})$ 。"""
    pk, _ = find_peaks(R, prominence=PROM)
    dσ = np.median(np.diff(s[pk]))
    return 1.0 / (2.0 * dσ * np.sqrt(N_SI ** 2 - np.sin(theta) ** 2))

def solve(path, theta, rng=FIT_RANGE):
    """一份附件: 式(30)拟合, 返回 (厚度/μm, |q|, rms, 参数, 节点, σ, R)。"""
    s, R = load_data(path)
    keep = R > 0 # 预处理: 删除反射率非正的点
    s, R = s[keep], R[keep]
    if rng:
        msk = (s >= rng[0]) & (s <= rng[1])
        s, R = s[msk], R[msk]

    r01 = (N_SI - N0) / (N_SI + N0)
    knots = np.linspace(s.min(), s.max(), KNOTS)

    # 厚度初值: 由相邻峰间距估算, 再在其附近扫描残差取最小者精化;
    # 扫描时 B 暂取常数, 只用于定厚度, 不参与最终结果。
    d_init = d0_from_peaks(s, R, theta)
    b_const = (np.sqrt(R.max() / 100.) - np.sqrt(R.min() / 100.)) / 2.
    grid = np.linspace(0.9 * d_init, 1.1 * d_init, 201)
    d_scan = grid[np.argmin([np.mean((r_multi(s, d, knots,
                                                np.full(KNOTS, b_const),
                                                0.0, r01, theta) - R) ** 2)
                             for d in grid])]

    # 式(30)拟合: 参数为 d、B 的节点值与 q;
    # B 节点初值取包络振幅的典型值。
    x0 = np.concatenate([d_scan, np.full(KNOTS, b_const), [0.05]])
    lo = np.concatenate([D_BOUND[0], np.zeros(KNOTS), [-0.3]])
    hi = np.concatenate([D_BOUND[1], np.full(KNOTS, 1.0), [0.3]])
    x, rms = fit(s, R, theta,
                 lambda sg, d, *rest: r_multi(sg, d, knots, rest[:KNOTS],
                                                rest[KNOTS], r01, theta),
                 x0, (lo, hi))
    return x[0] * 1e4, abs(x[-1]), rms, x, knots, s, R

def main():
    files = [('附件 3 (Si, 10°)', '题目/附件/附件3.xlsx', np.radians(10.)),
             ('附件 4 (Si, 15°)', '题目/附件/附件4.xlsx', np.radians(15.))]

    print('=' * 72)
    print(f'硅外延层厚度 (方案一: 多光束模型式(30)拟合; n = 3.42, '
          f'有效区间 {FIT_RANGE[0]:.0f}~{FIT_RANGE[1]:.0f} cm-1)')
    print('=' * 72)
    ds = []
    for name, path, theta in files:
        d, q, rms, _x, _knots, _s, _R = solve(path, theta)

```

```

        ds.append(d)
        print(f'{name:<16} d = {d:6.3 f} μm | q| = {q:.3 f} | rms = {rms:5.2 f}%')
    rel = abs(ds[0] - ds[1]) / np.mean(ds) * 100
    print('-' * 72)
    print(f'双入射角一致性: {ds[0]:.3 f} μm 与 {ds[1]:.3 f} μm, '
          f'相对差异 {rel:.2 f}%; 平均 d ≈ {np.mean(ds):.3 f} μm')
    print('=' * 72)

if __name__ == '__main__':
    main()

```

5.3 画图.py

```

# -*- coding: utf-8 -*-
"""

```

问题三结果可视化 —— 独立绘图脚本

生成论文 5.3 节的插图：

图13 两光束与多光束干涉条纹对比示意图（5.3.3）

图14 方案一（Drude 色散模型）全谱拟合结果对比（拟合曲线与残差，附件 3、4，5.3.5）

图15 反射谱的傅里叶功率谱（基频与谐波，附件 3、4，5.3.5）

注：硅光谱绘图函数 `plot_si_spectra` 对应旧图11，已从正文移除，默认不再调用；

图11 现为问题三求解流程图（Graphviz dot 生成）。

判别结果绘图函数 `plot_discrimination` 对应旧图13，已从正文移除，默认不再调用；

图13 现为两光束与多光束干涉条纹对比。

判别指标（与论文 5.3.2 一致）：

η —— 相邻峰谷之间位于中线以下的数据点占比：两光束正弦条纹 $\approx 50\%$ ，

多光束（谷尖深、峰宽平）显著低于 50%；

ρ —— 相邻出射光束振幅比 $\rho = |r_{10} \cdot r_{12} \cdot A|$ ，由包络线法反推的界面反射系数 r_{12} 与空气/外延层界面系数 r_{10} 相乘得到（式(28)）。

用法：

`python 5.3画图.py` # 输出到 .././ 图片/（复现2/图片/）

`python 5.3画图.py DIR` # 输出到指定目录

运行环境：Python 3 + numpy / scipy / openpyxl / matplotlib

```

"""
import sys
from pathlib import Path
import numpy as np
import openpyxl
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy.signal import find_peaks

def _load_checker():
    """按文件名加载 5.3检验.py（文件名以数字开头，不能用普通 import）。"""
    import importlib.util
    p = Path(__file__).resolve().parent / '5.3检验.py'
    spec = importlib.util.spec_from_file_location('q3_check', p)
    mod = importlib.util.module_from_spec(spec)

```

```

sys.modules['q3_check'] = mod
spec.loader.exec_module(mod)
return mod

_ck = _load_checker()

def _load_drude():
    """按文件名加载 5.3Drude求解.py（文件名以数字开头，不能用普通 import）。"""
    import importlib.util
    p = Path(__file__).resolve().parent / '5.3Drude求解.py'
    spec = importlib.util.spec_from_file_location('q3_drude', p)
    mod = importlib.util.module_from_spec(spec)
    sys.modules['q3_drude'] = mod
    spec.loader.exec_module(mod)
    return mod

_du = _load_drude()

# 字体设置：中文 SimSun（宋体），西文/数字 Times New Roman，数学 stix
plt.rcParams['font.family'] = ['Times New Roman', 'SimSun']
plt.rcParams['mathtext.fontset'] = 'stix'
plt.rcParams['axes.unicode_minus'] = False

BASE_DIR = Path(__file__).resolve().parent.parent.parent # 复现2 根目录
PROM = 1.0 # 峰谷检测显著度门槛（百分点），与 5.2 一致

# 附件：名称 -> (路径, 外延层折射率名义值, 残余射线带(剔除后参与统计))
FILES = {
    '附件1 (SiC, 10°)': ('题目/附件/附件1.xlsx', 2.55, (787., 968.)),
    '附件2 (SiC, 15°)': ('题目/附件/附件2.xlsx', 2.55, (787., 968.)),
    '附件3 (Si, 10°)': ('题目/附件/附件3.xlsx', 3.42, None),
    '附件4 (Si, 15°)': ('题目/附件/附件4.xlsx', 3.42, None),
}
SI_COLOR = {'附件3 (Si, 10°)': '#1f77b4', '附件4 (Si, 15°)': '#d62728'}
ZOOM = {'附件3 (Si, 10°)': (1400., 1800.), '附件4 (Si, 15°)': (1300., 1700.)}

def load(path):
    """读取 xlsx 附件，返回 (波数  $\text{cm}^{-1}$ , 反射率 %)。"""
    wb = openpyxl.load_workbook(path, read_only=True)
    rows = list(wb.active.iter_rows(values_only=True))
    wb.close()
    arr = np.array([[float(r[0]), float(r[1])] for r in rows[1:]
                    if r[0] is not None and r[1] is not None])
    return arr[:, 0], arr[:, 1]

def discriminate(s, R, n1, restrahlen):
    """逐段统计低于中线占比  $\eta$  与振幅比  $\rho$ ，返回 ( $\eta$ 均值,  $\rho$ 中位数, 段数)。

     $\eta$ : 相邻峰谷之间 R 低于中线  $(R_{\max}+R_{\min})/2$  的点占比；
     $\rho$ : 由包络振幅  $B=(\sqrt{R_{\max}}-\sqrt{R_{\min}})/2$  反推
         $r_{12} = B/(t_{01} \cdot t_{10})$ ,  $\rho = |r_{10} \cdot r_{12}|$  (吸收衰减  $A \approx 1$ )。
    """

```

```

#####
m = R > 0
if restrahlen :
    m &= ~(s > restrahlen [0]) & (s < restrahlen [1])
s, R = s[m], R[m]
pk, _ = find_peaks(R, prominence=PROM)
va, _ = find_peaks(-R, prominence=PROM)
E = sorted( list (pk) + list (va))
n0 = 1.0
r10 = (n1 - n0) / (n1 + n0)
t01, t10 = 2 * n0 / (n0 + n1), 2 * n1 / (n1 + n0)
fracs, rhos = [], []
for a, b in zip(E[:-1], E[1:]):
    seg = (s > s[a]) & (s < s[b])
    if seg.sum() < 20:
        continue
    mid = (R[a] + R[b]) / 2.
    fracs.append((R[seg] < mid).mean())
    Rmax, Rmin = max(R[a], R[b]), min(R[a], R[b])
    B = (np.sqrt(Rmax / 100.) - np.sqrt(Rmin / 100.)) / 2.
    rhos.append(r10 * B / (t01 * t10))
return np.mean(frac), np.median(rhos), len( fracs )

# ----- 硅光谱（旧图11，已移出正文，备用）
def plot_si_spectra ( out_dir ):
    """附件3、附件4的反射光谱全谱，上方各置一个放大子图展示局部
    条纹，直观呈现晶圆干涉条纹“谷尖深、峰宽平”的多光束特征。"""
    fig, ax = plt.subplots( figsize=(8, 4.8))
    zin_pos = {'附件3 (Si, 10°)': [0.14, 0.44, 0.34, 0.50],
               '附件4 (Si, 15°)': [0.56, 0.44, 0.34, 0.50]}
    for name, (path, _n1, _rb) in FILES.items():
        if name not in SI_COLOR:
            continue
        s, R = load(str(BASE_DIR / path))
        m = R > 0
        ax.plot(s[m], R[m], lw=0.7, color=SI_COLOR[name], label=name, zorder=3)
        # 局部放大子图：标注峰谷位置以展示峰形
        zlo, zhi = ZOOM[name]
        zm = m & (s >= zlo) & (s <= zhi)
        zin = ax.inset_axes(zin_pos[name])
        zin.plot(s[zm], R[zm], lw=0.8, color=SI_COLOR[name], zorder=3)
        pk, _ = find_peaks(R[zm], prominence=PROM)
        va, _ = find_peaks(-R[zm], prominence=PROM)
        zin.plot(s[zm][pk], R[zm][pk], '^', ms=3.2, color='tab:red', zorder=4)
        zin.plot(s[zm][va], R[zm][va], 'v', ms=3.2, color='tab:green', zorder=4)
        # 相邻峰谷对的中线：可见大部分数据点位于中线上方（η<50%）
        E = sorted( list (pk) + list (va))
        for a, b in zip(E[:-1], E[1:]):
            mid = (R[zm][a] + R[zm][b]) / 2.
            zin.hlines(mid, s[zm][a], s[zm][b], color='0.3', lw=0.8,
                       ls='--', zorder=2)
        zin.set_xlim(zlo, zhi)
        zin.set_xticks(np.linspace(zlo, zhi, 3))
        zin.tick_params( labelsize=7)
        zin.grid(alpha=0.3)

```

```

        zin.set_title(f'{name.split("(")[0]}局部放大', fontsize=8)
    ax.set_xlabel('波数  $\sigma$  /  $\text{cm}^{-1}$ ')
    ax.set_ylabel('反射率 R / %')
    ax.set_xlim(400, 4000)
    ax.legend(fontsize=9, loc='lower left')
    ax.grid(alpha=0.3)
    fig.tight_layout()
    _save(fig, out_dir, '硅晶圆片的反射光谱（附件3、附件4，未入文）')

```

----- 图 13

```

def plot_model_comparison(out_dir):
    """图13：两光束与多光束干涉条纹对比示意图。取硅的界面参数合成曲线；
    为清晰展示形态差异， $\rho$  取 0.20（大于硅晶圆实际值  $\approx 0.05$ ，差异方向相同）。
    左：两条曲线全谱对比；右：放大一个峰谷区间并绘出多光束曲线的中线。"""
    r01, B = -0.548, 0.083 # 硅:  $|r_{01}|=0.548$  ( $n=3.42$ )，B 由  $R_{\max}\approx 39\%$ 、 $R_{\min}\approx 21\%$  定
    g = 0.20 # 放大的振幅比，仅用于示意
    # 用波数作横轴:  $d=3.7\mu\text{m}$ 、 $n=3.42$ 、 $\theta=10^\circ$ ，条纹周期与附件 3 相近
    k = 4 * np.pi * 3.7e-4 * np.sqrt(3.42**2 - np.sin(np.radians(10))**2)
    s = np.linspace(1200., 2000., 8001)
    phi = k * s
    R2 = np.abs(r01 + B * np.exp(1j * phi))**2 * 100
    Rm = np.abs(r01 + B * np.exp(1j * phi)
                / (1 - g * np.exp(1j * phi)))**2 * 100

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 4.0))
    ax1.plot(s, R2, lw=1.1, color='0.45', label='两光束模型（式(5)）')
    ax1.plot(s, Rm, lw=1.1, color='#d62728', label='多光束模型（式(30)）')
    ax1.set_xlabel('波数  $\sigma$  /  $\text{cm}^{-1}$ ')
    ax1.set_ylabel('反射率 R / %')
    ax1.legend(fontsize=8)
    ax1.grid(alpha=0.3)

    # 右：取其中的一个谷-峰区间放大（谷变尖处），绘多光束曲线的中线与标注
    pk, _ = find_peaks(Rm, prominence=0.5)
    va, _ = find_peaks(-Rm, prominence=0.5)
    pairs = [(i, pk[pk > i][0]) for i in va if (pk > i).any()]
    a, b = pairs[len(pairs) // 2]
    ax2.plot(s[a:b], Rm[a:b], lw=1.1, color='#d62728', label='多光束模型')
    ax2.plot(s[a:b], R2[a:b], lw=1.1, ls='--', color='0.45', label='两光束模型')
    mid = (Rm[a] + Rm[b]) / 2
    ax2.hlines(mid, s[a], s[b], color='0.3', lw=0.8, ls=':', label='中线')
    ax2.set_xlim(s[a], s[b])
    ax2.set_xlabel('波数  $\sigma$  /  $\text{cm}^{-1}$ ')
    ax2.set_ylabel('反射率 R / %')
    ax2.legend(fontsize=8, loc='upper right')
    ax2.grid(alpha=0.3)
    fig.tight_layout()
    _save(fig, out_dir, '图13 两光束与多光束干涉条纹对比')

```

----- 旧图13（已移出正文，备用）

```

def plot_discrimination(stats, out_dir):
    """多光束干涉定量判别（旧图13，已移出正文）——左图为低于中线占比  $\eta$ （两光束 $\approx 50\%$ ），
    右图为振幅比  $\rho$ （ $\rho \leq 0.03$  两光束、 $\rho \geq 0.05$  多光束显著）。"""
    names = list(stats.keys())

```

```

eta = np.array([ stats[k][0] * 100 for k in names])
rho = np.array([ stats[k][1] for k in names])
cols = ['#7f7f7f', '#7f7f7f', '#1f77b4', '#d62728']
x = np.arange(len(names))

fig, axes = plt.subplots(1, 2, figsize=(9, 4.0))
ax = axes[0]
bars = ax.bar(x, eta, width=0.55, color=cols, edgecolor='k', lw=0.5, zorder=3)
ax.axhline(50, color='k', lw=1.0, ls='—', label='两光束干涉参考线 50%')
ax.axhspan(48, 52, color='#cde7c0', alpha=0.5, label='两光束容差 ±2%')
for b, v in zip(bars, eta):
    ax.text(b.get_x() + b.get_width() / 2, v + 0.7, f'{v:.1f}%',
            ha='center', fontsize=8)
ax.set_ylabel('低于中线占比  $\eta$  / %')
ax.set_ylim(0, 60)

ax = axes[1]
bars = ax.bar(x, rho, width=0.55, color=cols, edgecolor='k', lw=0.5, zorder=3)
ax.axhline(0.03, color='#2ca02c', lw=1.0, ls='—', label='两光束上限  $\rho=0.03$ ')
ax.axhline(0.05, color='#d62728', lw=1.0, ls='—', label='多光束显著  $\rho=0.05$ ')
for b, v in zip(bars, rho):
    ax.text(b.get_x() + b.get_width() / 2, v + 0.001, f'{v:.3f}',
            ha='center', fontsize=8)
ax.set_ylabel('振幅比  $\rho$ ')
ax.set_ylim(0, 0.07)

for ax in axes:
    ax.set_xticks(x)
    ax.set_xticklabels(names, fontsize=7.5, rotation=15)
    ax.legend(fontsize=7, loc='upper right')
    ax.grid(alpha=0.3, axis='y')
fig.tight_layout()
_save(fig, out_dir, '多光束干涉的定量判别结果（未入文）')

```

————— 图 14

```

def plot_multibeam_fit(name, ang, path, theta, out_dir):
    """图14 (5.3.5)：方案一（Drude 色散模型）全谱拟合结果对比。
    实测曲线取全谱（预处理后），拟合曲线由衬底 Drude 介电函数代入
    多光束反射率公式拟合所得参数画出，残差绘于右轴。"""
    s, R = _du.load_data(str(BASE_DIR / path))
    m = R > 0
    s, R = s[m], R[m]
    x, rms = _du.fit_drude(s, R, theta)
    Rm = _du.R_model(s, *x, theta)

    fig, ax = plt.subplots(figsize=(8, 4.8))
    ax.plot(s, R, lw=0.9, color='0.35', label='实测反射率')
    ax.plot(s, Rm, lw=1.3, ls='—', color='#d62728',
            label='Drude 色散模型拟合')
    ax.set_xlabel('波数  $\sigma$  /  $\text{cm}^{-1}$ ')
    ax.set_ylabel('反射率  $R$  / %')
    ax.set_xlim(s.min(), s.max())
    ax.legend(fontsize=9,
            title=f'd={x[0] * 1e4:.2f}  $\mu\text{m}$ ,  $\sigma_p$ = $\{x[2]:.0f\}$   $\text{cm}^{-1}$ , '
            f' $\gamma$ = $\{x[3]:.0f\}$   $\text{cm}^{-1}$ , 残差 rms= $\{rms:.2f\}$  %'),

```

```

        title_fontsize =9)
ax.grid(alpha=0.3)
# 残差子图（右侧纵轴，浅色弱化，与图8同风格）
ax2 = ax.twinx()
ax2.plot(s, R - Rm, lw=0.7, color='#c8c8c8')
ax2.set_ylabel('残差  $R_{obs} - R_{fit}$  / %', color='#a8a8a8')
ax2.tick_params(axis='y', labelcolor='#a8a8a8')
ax2.axhline(0, color='#dcdcdc', lw=0.6, ls=':')
rmax = np.max(np.abs(R - Rm)) * 1.3
ax2.set_ylim(-rmax, rmax)
fig.tight_layout()
_save(fig, out_dir, f'图14 方案一拟合结果（{name.split("(")[0]}, {ang:.0f}°）')

# ----- 图 15
def plot_fft_spectrum(name, ang, path, theta, out_dir):
    """图15（5.3.5）：截断区间内反射谱的傅里叶功率谱。
    标出基频与前两次谐波，多光束干涉的谐波结构清晰可见。"""
    s, R = _ck.load_data(str(BASE_DIR / path))
    f, power, f0 = _ck.fft_spectrum(s, R)
    d = _ck.fft_thickness(s, R, theta)[0]
    m = f <= 4 * f0 # 只画到四次基频以内
    f, power = f[m] * 1e3, power[m] # 频率刻度  $\times 10^3$  周期/cm-1
    power = power / power.max()

    fig, ax = plt.subplots(figsize=(8, 4.4))
    ax.plot(f, power, lw=1.1, color='#1f77b4')
    ax.fill_between(f, power, alpha=0.12, color='#1f77b4')
    for k, lab in ((1, '基频 f0'), (2, '二次谐波'), (3, '三次谐波')):
        ax.axvline(k * f0 * 1e3, color='#d62728', lw=0.9, ls='—')
        ax.annotate(lab, (k * f0 * 1e3, 0.98), xytext=(k * f0 * 1e3 + 0.12, 0.93),
                    fontsize=9, color='#d62728')
    ax.set_xlabel('频率  $f / (10^{-3} \text{ 周期} \cdot \text{cm}^{-1})$ ')
    ax.set_ylabel('归一化功率')
    ax.set_xlim(0, 4 * f0 * 1e3)
    ax.set_ylim(0, 1.08)
    ax.annotate(f'基频 f0={f0 * 1e3:.2f}, 对应厚度 d={d:.2f} μm',
                xy=(0.03, 0.86), xycoords='axes fraction',
                ha='left', fontsize=9.5,
                bbox=dict(fc='white', ec='#c8c8c8', lw=0.6, alpha=0.9))
    ax.grid(alpha=0.3)
    fig.tight_layout()
    _save(fig, out_dir,
          f'图15 反射谱的傅里叶功率谱（{name.split("(")[0]}, {ang:.0f}°）')

# ----- 工具
def _save(fig, out_dir, name):
    """同时保存 svg（论文嵌入）与 png（快速预览）。"""
    fig.savefig(out_dir / f'{name}.svg', dpi=300)
    fig.savefig(out_dir / f'{name}.png', dpi=200)
    plt.close(fig)
    print(f'已保存 {out_dir / name}.svg')

def main():

```



```

out_dir = Path(sys.argv[1]) if len(sys.argv) > 1 else BASE_DIR / '图片'
out_dir.mkdir(parents=True, exist_ok=True)
print(f'问题三结果可视化：输出目录 {out_dir}')

stats = {}
print(f'附件':<18}{ '峰谷段数':>8}{ '低于中线占比':>12}{ '振幅比p':>10}')
for name, (path, n1, rb) in FILES.items():
    s, R = load(str(BASE_DIR / path))
    eta, rho, nseg = discriminate(s, R, n1, rb)
    stats[name] = (eta, rho, nseg)
    print(f' {name}:<20}{nseg:>8}{eta*100:>11.1f}%{rho:>10.3f}')

# 旧图11（硅光谱）已移出正文，默认不生成；如需重新插入请启用并按出现位置重新编号
# plot_si_spectra(out_dir)
plot_model_comparison(out_dir) # 图13（合成示意图，不依赖实测数据）
# 旧图13（判别结果）已移出正文，默认不生成；如需重新插入请启用并重新编号
# plot_discrimination(stats, out_dir)
si_files = {'附件3（Si, 10°）': ('题目/附件/附件3.xlsx', 10.),
            '附件4（Si, 15°）': ('题目/附件/附件4.xlsx', 15.)}
for name, (path, ang) in si_files.items():
    plot_multibeam_fit(name, ang, path, np.radians(ang), out_dir) # 图14
    plot_fft_spectrum(name, ang, path, np.radians(ang), out_dir) # 图15
print('全部插图生成完毕。')

if __name__ == '__main__':
    main()

```

5.3 检验.py

```

# -*- coding: utf-8 -*-
"""
问题三：硅外延层厚度结果的FFT检验（与论文 5.3.5 节一致）
=====
主结果为相邻峰谷法（5.3求解.py）。这里用快速傅里叶变换频谱分析作
独立检验：干涉条纹等价于反射率随波数以基频


$$f = 2 \cdot d \cdot c \quad (\text{周期数每 cm}^{-1})$$


振荡，对谱线作傅里叶变换后由功率谱主峰确定基频，即得


$$d = 1e4 \cdot f / (2 \cdot c) \text{ }\mu\text{m}。$$


处理步骤依次为：截取波数不低于 1500  $\text{cm}^{-1}$  的区间；插值到均匀网格；
三次多项式去趋势，扣除谱线的缓慢背景；乘汉宁窗抑制频谱泄漏；
零填充细化频率刻度后变换。多光束干涉使条纹波形偏离正弦，只在频谱上
产生整倍频的谐波，不移动基频，故该检验不受峰谷识别的影响。

数据预处理与 5.3.5 节相同：删除反射率非正的点；波数低于 1500  $\text{cm}^{-1}$ 
的低频段不参与统计。

运行环境：Python 3 + numpy / openpyxl
用法：在项目根目录运行 python "5 模型的建立与求解/5.3 问题3/5.3检验.py"
"""
import numpy as np

```

```

import openpyxl

N0 = 1.0
N_SI = 3.42          # 硅的文献名义折射率（红外透明区）
S_CUT = 1500.        # 低频截断波数（ $\text{cm}^{-1}$ ），与 5.3求解.py 一致
N_PAD = 8            # 零填充倍数，细化频率刻度

def load_data(path):
    """读取 xlsx 附件，返回 (波数  $\text{cm}^{-1}$ , 反射率 %)。”"""
    wb = openpyxl.load_workbook(path, read_only=True)
    rows = list(wb.active.iter_rows(values_only=True))
    wb.close()
    arr = np.array([[float(r[0]), float(r[1])] for r in rows[1:]
                    if r[0] is not None and r[1] is not None])
    return arr[:, 0], arr[:, 1]

def preprocess(s, R):
    """截断、均匀化、去趋势，返回 (均匀网格, 去趋势后的反射率)。”"""
    m = (R > 0) & (s >= S_CUT)
    s, R = s[m], R[m]
    sg = np.linspace(s.min(), s.max(), len(s))
    R = np.interp(sg, s, R)
    R = R - np.polyval(np.polyfit(sg, R, 3), sg) # 三次多项式去趋势
    return sg, R

def fft_spectrum(s, R):
    """对预处理后的谱线作加窗、零填充的 FFT，
    返回 (频率刻度/周期数每  $\text{cm}^{-1}$ , 功率, 基频)。”"""
    sg, R = preprocess(s, R)
    sp = np.fft.rfft(R * np.hanning(len(R)), n=N_PAD * len(R))
    df = 1.0 / ((sg[-1] - sg[0]) * N_PAD)
    f = np.arange(len(sp)) * df
    power = np.abs(sp)
    k = np.argmax(power[1:]) + 1          # 跳过零频
    return f, power, f[k]

def fft_thickness(s, R, theta):
    """FFT 检验给出的厚度（ $\mu\text{m}$ ）。”"""
    c = np.sqrt(N_SI ** 2 - (N0 * np.sin(theta)) ** 2)
    _f, _power, f0 = fft_spectrum(s, R)
    return 1e4 * f0 / (2 * c), f0

def main():
    files = [('附件 3 (Si, 10°)', '题目/附件/附件3.xlsx', np.radians(10.)),
             ('附件 4 (Si, 15°)', '题目/附件/附件4.xlsx', np.radians(15.))]

    print('=' * 70)
    print('硅外延层厚度结果的 FFT 检验（主结果：相邻峰谷法 3.41 / 3.40  $\mu\text{m}$ ）')
    print('=' * 70)
    for name, path, theta in files:
        s, R = load_data(path)

```

```

d, f0 = fft_thickness (s, R, theta)

print(f'{name:<16} d = {d:.3 f} μm 基频 f0 = {f0:.5 f} 周期/cm-1'
      f' (2f0 = {2 * f0:.5 f}, 3f0 = {3 * f0:.5 f}) ')

print('=' * 70)

if __name__ == '__main__':
    main()

```

5.3 判别.py

```

# -*- coding: utf-8 -*-
"""

```

问题三：多光束干涉的定量判别（与论文 5.3.3、5.3.4 节一致）

按 5.3.3 给出的两个互补判据对四份附件逐一统计：

判据一（振幅比，式(33)）：

第二束光振幅 $B = (\sqrt{R_{\max}} - \sqrt{R_{\min}})/2$ （问题一式(6)），
与式(28)对比反推衬底界面反射系数，
 $r_{12_hat} = B / (t_{01} * t_{10} * A)$ ，取 $A \approx 1$ ，
振幅比估计值 $\rho_{hat} = |r_{10}| * r_{12_hat}$ ，
其中 r_{10} 、 t_{01} 、 t_{10} 按垂直入射菲涅耳公式由外延层折射率名义值算出。

判据二（条纹形态，式(34)）：

第 j 对相邻峰、谷之间的中线取
 $R_{mid} = (R_{\max} + R_{\min})/2$ ，
统计区间内位于中线以下的数据点占比 η_j ，
对全部峰谷对取平均得 η 。

数据预处理与 5.2 节相同：删除 $R \leq 0$ 的异常点；峰、谷检测的显著度门槛
取 1.0 个百分点。附件 1、2 的统计中剔除两束光模型失效的残余射线带
787~968 cm^{-1} 。

运行环境：Python 3 + numpy / scipy / openpyxl

用法：在项目根目录运行 `python "5 模型的建立与求解/5.3 问题3/5.3判别.py"`

```

import numpy as np
from scipy.signal import find_peaks
import openpyxl

N0 = 1.0          # 空气折射率
PROM = 1.0        # 峰谷检测显著度门槛（百分点），与 5.2 节相同
MIN_PTS = 20      # 参与统计的峰谷区间最少数据点数
RESTR = (787.0, 968.0) # 碳化硅残余射线带（附件 1、2 剔除）
RHO_LO, RHO_HI = 0.03, 0.05 # 振幅比阈值（数值模拟确定，见 5.3.3）
ETA_TOL = 0.02    # eta 的正常波动半径（50%±2%）

def load_data(path):
    """读取 xlsx 附件，返回(波数 cm-1, 反射率 %)。"""
    wb = openpyxl.load_workbook(path, read_only=True)
    rows = list(wb.active.iter_rows(values_only=True))
    wb.close()
    arr = np.array([[float(r[0]), float(r[1])] for r in rows[1:]])

```

```

        if r[0] is not None and r[1] is not None])
    return arr[:, 0], arr[:, 1]

def discriminate (path, n1, restr=None):
    """对一份附件统计 (峰谷对数, eta, rho_hat)。"""
    s, R = load_data(path)
    keep = R > 0                                # 预处理规则□
    if restr :
        keep &= ~(s > restr [0]) & (s < restr [1])
    s, R = s[keep], R[keep]

    pk, _ = find_peaks(R, prominence=PROM)
    va, _ = find_peaks(-R, prominence=PROM)
    E = sorted( list (pk) + list (va))

    # 垂直入射菲涅耳系数 (名义折射率)
    r10 = (n1 - N0) / (n1 + N0)                # 膜内→空气
    t01 = 2 * N0 / (N0 + n1)                   # 空气→膜
    t10 = 2 * n1 / (n1 + N0)                   # 膜→空气

    etas, rhos = [], []
    for a, b in zip(E[:-1], E[1:]):
        seg = (s > s[a]) & (s < s[b])
        if seg.sum() < MIN_PTS:
            continue
        Rmax, Rmin = max(R[a], R[b]), min(R[a], R[b])
        mid = (Rmax + Rmin) / 2.                # 中线, 式(34)前的定义
        etas.append((R[seg] < mid).mean())
        B = (np.sqrt(Rmax / 100.) - np.sqrt(Rmin / 100.)) / 2. # 问题一式(6)
        r12_hat = B / (t01 * t10)              # 取 A≈1, 式(33)
        rhos.append(abs(r10) * r12_hat)
    return len(etas), np.mean(etas), np.median(rhos)

def judge(eta, rho):
    """按 5.3.3 的阈值给出判别结论。"""
    if rho <= RHO_LO and abs(eta - 0.5) <= ETA_TOL:
        return '两光束模型适用'
    if rho >= RHO_HI or abs(eta - 0.5) > ETA_TOL:
        return '存在多光束干涉'
    return '介于阈值之间, 需结合数据检验'

def main():
    files = [('附件 1 (SiC, 10°)', '题目/附件/附件1.xlsx', 2.55, RESTR),
             ('附件 2 (SiC, 15°)', '题目/附件/附件2.xlsx', 2.55, RESTR),
             ('附件 3 (Si, 10°)', '题目/附件/附件3.xlsx', 3.42, None),
             ('附件 4 (Si, 15°)', '题目/附件/附件4.xlsx', 3.42, None)]

    print('=' * 72)
    print('多光束干涉定量判别: 峰值段数 / 低于中线占比 eta / 振幅比 rho_hat')
    print('=' * 72)
    for name, path, n1, restr in files :
        J, eta, rho = discriminate (path, n1, restr )
        print(f'{name:<18} 峰谷段数 {J:>3} ',

```

```

        f`eta = {eta*100:5.1f}%   rho_hat = {rho:.3 f}   '
        f`→ {judge(eta, rho)}')
    print('=' * 72)

if __name__ == '__main__':
    main()

```

5.3 判别模拟.py

```

# -*- coding: utf-8 -*-
"""5.3.3 两光束模型数值模拟（对应正式文(34)附近的判别阈值）。

模拟内容：
1. 按硅的参数合成两光束反射谱，考察多光束分母因子（振幅比  $\rho$  取
   0.03、0.05、0.10）给峰谷反射率读数、反推折射率与  $\eta$  带来的改变，
   用于确定式(33)的两个阈值；
2. 在两光束谱上叠加与实测相当水平的扰动，重复统计  $\eta$ ，
   用于给出“ $\eta$  正常波动范围约 50%±2%”的结论。

运行环境：Python 3 + numpy / scipy
用法：在项目根目录运行 python "5 模型的建立与求解/5.3 问题3/5.3判别模拟.py"
"""

import numpy as np
from scipy.signal import find_peaks

# 硅参数：|r01| = 0.548（n = 3.42），B 由实测峰谷反射率
# Rmax ≈ 39%、Rmin ≈ 21% 确定；入射角 10°，厚度 3.7 μm。
R01, B = -0.548, 0.083
N1, THETA, D = 3.42, np.radians(10.), 3.7e-4
C = np.sqrt(N1**2 - np.sin(THETA)**2)
S = np.linspace(600., 3000., 24001)
PHI = 4 * np.pi * D * C * S

def R_of(rho):
    """多光束反射率， $\rho=0$  时退化为两光束。"""
    return np.abs(R01 + B * np.exp(1j * PHI)
                  / (1 - rho * np.exp(1j * PHI)))**2 * 100

def eta_of(R):
    """全部峰谷区间低于中线的点占比  $\eta$ （式(34)）。"""
    pk, _ = find_peaks(R, prominence=0.5)
    va, _ = find_peaks(-R, prominence=0.5)
    E = sorted(list(pk) + list(va))
    fr = [(R[a:b] < (R[a] + R[b]) / 2).mean()
           for a, b in zip(E[:-1], E[1:])] if b - a > 50]
    return np.mean(fr) * 100

def main():
    print('一、振幅比  $\rho$  的影响（无扰动）')
    print('=' * 66)
    R_base = R_of(0.0)
    pk0, _ = find_peaks(R_base, prominence=0.5)

```

```

va0, _ = find_peaks(-R_base, prominence=0.5)
Rmax0, Rmin0 = R_base[pk0].mean(), R_base[va0].mean()
u0 = (np.sqrt(Rmax0 / 100) + np.sqrt(Rmin0 / 100)) / 2
print(f'基准（两光束）: Rmax = {Rmax0:.2f}% Rmin = {Rmin0:.2f}% '
      f'η = {eta_of(R_base):.1f}%')
for rho in (0.03, 0.05, 0.10):
    R = R_of(rho)
    pk, _ = find_peaks(R, prominence=0.5)
    va, _ = find_peaks(-R, prominence=0.5)
    Rmax, Rmin = R[pk].mean(), R[va].mean()
    u = (np.sqrt(Rmax / 100) + np.sqrt(Rmin / 100)) / 2
    print(f'ρ = {rho:.2f}: Rmax {Rmax - Rmax0:+.2f} 个百分点, '
          f'Rmin {Rmin - Rmin0:+.2f} 个百分点, '
          f'反推折射率偏差 {(u - u0) / u0 * 100:+.2f}%, '
          f'η = {eta_of(R):.1f}%')

print()
print('二、两光束谱叠加扰动后 η 的波动（各 200 次）')
print('-' * 66)
rng = np.random.default_rng(0)
for sig in (0.05, 0.1):
    vals = [eta_of(R_base + rng.normal(0., sig, len(R_base)))
            for _ in range(200)]
    vals = np.array(vals)
    print(f'扰动 {sig:.2f} 个百分点: η 在 '
          f'{vals.min():.1f}% ~ {vals.max():.1f}% 之间 '
          f'（标准差 {vals.std():.2f} 个百分点）')

if __name__ == '__main__':
    main()

```

5.3 求解.py

```

# -*- coding: utf-8 -*-
"""

```

问题三：硅外延层厚度的计算（与论文 5.3.5 节一致）

附件 3、4 已确认存在多光束干涉。多光束干涉的分母因子以 2π 为周期，峰谷位置与两光束情形一致、条纹周期不变，因此仍按问题一的方案二（相邻峰谷法）由峰谷波数求厚度；但峰谷反射率读数在多光束干涉下有偏，不能再由包络线法反推折射率，折射率改取硅的文献名义值 $n = 3.42$ ，逐对代入问题一式(5)：

$$d_j = 1 / (4 \cdot |\sigma_v \cdot c - \sigma_p \cdot c|), \quad c = \sqrt{n^2 - n_0^2 \cdot \sin^2 \theta}.$$

数据预处理与 5.2 节相同：删除反射率非正的点；峰、谷检测显著度门檻取 1.0 个百分点。此外，波数低于约 1500 cm^{-1} 的低频段，重掺杂衬底的折射率偏离名义值较多，常数折射率的前提不再成立，该段的峰谷对不参与统计。

运行环境：Python 3 + numpy / scipy / openpyxl

用法：在项目根目录运行 `python "5 模型的建立与求解/5.3 问题3/5.3求解.py"`

```

"""

```

```

import numpy as np

```

```

from scipy.signal import find_peaks
from scipy.stats import median_abs_deviation
import openpyxl

N0 = 1.0
N_SI = 3.42          # 硅的文献名义折射率（红外透明区）
PROM = 1.0           # 峰谷检测显著度门槛（百分点），与 5.2 节相同
D_BOUND = (0.5, 30.0) # 峰谷对厚度的合理范围（ $\mu\text{m}$ ）
S_CUT = 1500.        # 低频截断波数（ $\text{cm}^{-1}$ ），低于该值的峰谷对不参与统计

def load_data(path):
    """读取xlsx 附件，返回(波数  $\text{cm}^{-1}$ , 反射率 %)。"""
    wb = openpyxl.load_workbook(path, read_only=True)
    rows = list(wb.active.iter_rows(values_only=True))
    wb.close()
    arr = np.array([[float(r[0]), float(r[1])] for r in rows[1:]
                    if r[0] is not None and r[1] is not None])
    return arr[:, 0], arr[:, 1]

def solve(path, theta):
    """一份附件：峰谷检测 → 逐对厚度 → 筛选，返回峰谷对数组。

    数组列为 [ $\sigma_1$ ,  $\sigma_2$ ,  $d_j/\mu\text{m}$ ]， $\sigma_1$ 、 $\sigma_2$  为该对两极值按波数排列。
    """
    s, R = load_data(path)
    keep = R > 0          # 预处理：删除反射率非正的点
    s, R = s[keep], R[keep]
    pk, _ = find_peaks(R, prominence=PROM)
    va, _ = find_peaks(-R, prominence=PROM)
    E = sorted([(s[i], R[i], 'max') for i in pk]
               + [(s[i], R[i], 'min') for i in va])
    c = np.sqrt(N_SI ** 2 - (N0 * np.sin(theta)) ** 2)
    rows = []
    for a, b in zip(E[:-1], E[1:]):
        if a[2] == b[2]:          # 必须峰谷相邻
            continue
        s1, s2 = min(a[0], b[0]), max(a[0], b[0])
        if min(s1, s2) < S_CUT:
            continue          # 低频段折射率偏离名义值，截断
        d_j = 1e4 / (4.0 * c * (s2 - s1))          #  $\text{cm} \rightarrow \mu\text{m}$ 
        if not (D_BOUND[0] <= d_j <= D_BOUND[1]):
            continue          # 谱线起突处的孤立极值
        rows.append((s1, s2, d_j))
    return np.array(rows)

def main():
    files = [('附件 3 (Si, 10°)', '题目/附件/附件3.xlsx', np.radians(10.)),
             ('附件 4 (Si, 15°)', '题目/附件/附件4.xlsx', np.radians(15.))]

    print('=' * 66)
    print(f'硅外延层厚度（相邻峰谷法，n = {N_SI}）')
    print('=' * 66)
    meds = []

```

```

for name, path, theta in files :
    P = solve(path, theta)
    d_j = P[:, 2]
    med = np.median(d_j)
    mad = median_abs_deviation(d_j, scale=1.0)
    meds.append(med)
    print(f'\n 【{name}】 ')
    print(f'   {"σ1":>9} {"σ2":>9} {"d_j/μm":>8}')
    for s1, s2, dj in P:
        print(f'   {s1:9.2 f} {s2:9.2 f} {dj:8.3 f}')
    print(f'   峰谷对 {len(P)} 对, 中位数 {med:.2f} μm, '
          f'MAD {mad:.2f} μm ( {mad / med * 100:.1f}% ) ')
    rel = abs(meds[0] - meds[1]) / np.mean(meds) * 100
    print('\n' + '-' * 66)
    print(f'双入射角一致性: {meds[0]:.2f} μm 与 {meds[1]:.2f} μm, '
          f'相对差异 {rel:.1 f}%')
    print(f'最终结果: 硅外延层厚度 d ≈ {np.mean(meds):.2f} μm')
    print('=' * 66)

if __name__ == '__main__':
    main()

```